



Ecole Nationale Supérieure d'Informatique et d'Analyse des Systèmes

Rapport de stage de fin de deuxième année

Explorateur de Dépendances BigQuery : Gestion des Autorisations et Visualisation des Dépendances

Filière : Génie de la data (GD)

Réalisé par:

Meriem Sabour

Encadré par:

Pr. Anas Mastour

Soutenu le 3 novembre, Devant le jury composé de :

Pr. Boutaina HDIOUD

Pr. Rachid OULAD HAJ THAMI

-ANNÉE UNIVERSITAIRE : 2023 – 2024-

Remerciement

Au terme de ce travail, je profite de l'occasion pour remercier du fond du cœur tous ceux qui ont contribué de près ou de loin à en faire de ce stage une expérience enrichissante, j'adresse donc, en particulier mes sincères remerciements à Mr Anass MASTOUR pour avoir su m'éclairer sur les bonnes solutions lors des débuts délicats de notre projet, pour les conseils qu'il a pu me prodiguer tout au long de cette période, et pour son suivi continu de mon travail ainsi que son effort pour nous fournir la documentation et les données nécessaires du sujet. Je tiens à le remercier pour sa patience lors de mon introduction dans le côté métier du projet, sa générosité dans le partage d'information à propos de son métier comme un Devops engineer au sein de SOFRECOM.

Je tiens également à remercier mon binôme de stage Hamid BOUARGALNE pour sa collaboration exceptionnelle tout au long de notre période de travail et son engagement, Je tiens à exprimer ma sincère gratitude aussi envers l'ensemble des membres de l'équipe SOLID de m'avoir si bien accueilli. Grâce à tous, j'ai pu effectuer les tâches qui m'ont été confiées dans des conditions optimales.

Je tiens aussi à remercier notre encadrante Mme Sanaa EL FKIHI pour sa disponibilité et ses précieux conseils.

J'exprime également ma gratitude aux membres du jury, qui nous ont honorés en acceptant de juger ce travail.

Résumé

Ce rapport est le fruit de mon travail dans le cadre de mon stage effectué chez SOFRECOM, où j'ai contribué à la conception et à la mise en œuvre d'un système de gestion des autorisations et de visualisation des dépendances dans Google BigQuery. Ce projet représente une opportunité précieuse qui m'a permis d'approfondir mes compétences techniques et de mettre en pratique de nouvelles technologies dans un environnement professionnel.

Au cours de mon stage, j'ai eu l'occasion de plonger dans le contexte du projet "Explorateur de Dépendances BigQuery". Mon travail a consisté à comprendre en détail les besoins du projet, à développer des solutions pour la gestion des autorisations d'accès aux données, et à mettre en place un système de visualisation des dépendances entre les vues et les requêtes dans Google BigQuery.

Ce rapport présente une vue d'ensemble de SOFRECOM, la mission spécifique que j'ai accomplie au cours de mon stage, ainsi que les étapes clés de la réalisation du projet. Mon objectif principal était de contribuer à la création d'un outil précieux pour l'équipe, permettant une gestion efficace des autorisations et une visualisation claire des dépendances, en améliorant ainsi la performance et la sécurité du système.

Abstract

This report is the result of my work during my internship at SOFRECOM, where I contributed to the design and implementation of an authorization management and dependency visualization system in Google BigQuery. This project has been a valuable opportunity for me to deepen my technical skills and apply new technologies in a professional environment.

During my internship, I had the opportunity to delve into the context of the "BigQuery Dependency Explorer" project. My work involved a detailed understanding of the project's requirements, the development of solutions for data access authorization management, and the establishment of a system for visualizing dependencies between views and queries in Google BigQuery.

This report provides an overview of SOFRECOM, details of the specific mission I undertook during my internship, and the key stages of the project's implementation. My primary goal was to contribute to the creation of a valuable tool for the team, enabling efficient authorization management and clear dependency visualization, thereby improving system performance and security.

Table des matières

Introduction générale	9
1 Contexte du projet	10
1.1 Introduction:	11
1.2 Présentation de l'organisme	11
1.2.1 Présentation de SOFRECOM:	11
1.2.2 Principales missions:	11
1.2.3 Domaines d'expertise :	12
1.2.4 SOFRECOM en chiffres :	13
1.3 Présentation du projet	13
1.3.1 Problématique du projet:	13
1.3.2 Objectifs du projet:	14
1.4 Méthodologie du travail	15
1.4.1 Méthodologie du développement:	15
1.4.2 Etapes de planification du projet:	16
1.5 Conclusion:	16
2 Analyse et conception	17
2.1 Analyse de l'Existant	18
2.2 Spécification des besoins et exigences	18
2.3 Choix de dataset	20
2.4 Stockage dans Cloud Firestore	20
2.5 Diagrammes de Cas d'Utilisation et de Séquence	21
2.6 Architecture de la solution:	23
3 Réalisation	24
3.1 Introduction:	25
3.2 Technologies et outils utilisés:	25
3.2.1 Google Cloud Platform (GCP):	25
3.2.2 Google BigQuery:	25
3.2.3 Cloud Firestore:	26
3.2.4 Terraform:	26
3.2.5 FastApi:	27
3.2.6 CPython:	27
3.2.7 Angular:	27
3.3 Etapes de réalisation:	28
3.3.1 Choix de dataset:	28
3.3.2 Création des vues imbriquées:	29
3.3.3 Extraction des dépendances:	30
3.3.4 Transition vers les Microservices avec FastAPI:	32

3.3.5	Stockage des Demandes dans Cloud Firestore:	33
3.3.6	Accord des Droits avec Terraform:	35
3.4	Interface utilisateur:	37
3.5	Interface en ligne de commande:	43
3.6	Conclusion:	46
Conclusion générale		47
Webographie		48

Table des figures

1.1	domaines d'intervention de Sofrecom Maroc	13
1.2	Les chiffres clés de SOFRECOM	13
1.3	Une vue sur la répartition du projet sur Gitlab	15
1.4	Diagramme de Gantt	16
2.1	Structure du Stockage des demandes dans Firestore	21
2.2	Diagramme de cas d'utilisation	21
2.3	Diagramme de séquence: Visualiser les dépendances	22
2.4	Architecture de la solution proposée	23
3.1	Logo Google cloud platform	25
3.2	Logo BigQuery	25
3.3	Logo Firestore	26
3.4	Logo Terraform	26
3.5	Logo FastApi	27
3.6	Logo Python	27
3.7	Logo Angular	27
3.8	Plateforme GCP	28
3.9	Dataset choisie	29
3.10	Création d'une vue imbriquée	29
3.11	Création d'une vue plus complexe	29
3.12	Vue sur l'ensemble de données dans BigQuery	30
3.13	Résultat d'extraction des dépendances	31
3.14	Tests unitaires	31
3.15	La couverture du code	32
3.16	Microservices	33
3.17	Stockage des demandes dans Firestore	35
3.18	Fichier de Configuration Terraform	36
3.19	IAM BigQuery roles	36
3.20	Terraform requests	37
3.21	L'interface graphique	38
3.22	L'interface graphique 2	38
3.23	Les dépendances d'une vue	39
3.24	Les dépendances d'une requête	39
3.25	Créer une nouvelle demande	40
3.26	Création d'une demande	40
3.27	Une vue sur toutes les demandes	41
3.28	Valider une demande	41
3.29	Demande validée	42
3.30	Demande supprimée	42

3.31	Vérification des droits accordés.	43
3.32	Options Bqacm.	44
3.33	Générer dépendances d'une view.	45
3.34	Générer dépendances d'une query.	45

Introduction générale

À l'ère des données massives et des analyses complexes, l'adoption de Google Cloud Platform (GCP) par Sofrecom a été un choix stratégique pour moderniser leur infrastructure de données. GCP offre des avantages significatifs tels que l'évolutivité, la sécurité renforcée, la flexibilité et des outils d'analyse avancés. Cette transition a permis à Sofrecom d'améliorer l'efficacité opérationnelle en remplaçant les anciens outils par des solutions cloud innovantes. Parmi ces outils on cite Google Cloud BigQuery qui est un Warehouse pour stocker les données et exécuter des analyses sur de grandes quantités de données en quasi-temps réel.

À mesure que les volumes de données atteignent des proportions massives, l'analyse des requêtes et les vues BigQuery, impliquant l'extraction des dépendances de la vue incluant les tables et les vues imbriquées sous forme d'arbre, devient extrêmement complexe. D'un côté, il est crucial de comprendre les relations complexes entre les entités pour une analyse précise, y compris les dépendances indirectes à travers les vues imbriquées. D'un autre côté, la gestion manuelle des autorisations d'accès à ces données devient non seulement fastidieuse mais également extrêmement coûteuse en termes de temps et de ressources humaines. C'est dans ce contexte que notre mission de stage prend tout son sens

J'ai eu le privilège d'effectuer mon stage de deuxième année (PFA) au sein de SOFRECOM, une entreprise experte dans le conseil aux opérateurs télécoms sur les quatre continents. L'équipe dont je faisais partie, se consacre à la migration des données vers le cloud et à leur maintenance.

Ma mission spécifique au sein de cette équipe était de mettre en place un système automatisé pour gérer efficacement les autorisations d'accès aux tables BigQuery, tout en extrayant systématiquement l'arborescence des dépendances des vues. Cette automatisation visait à améliorer l'efficacité opérationnelle en réduisant les coûts liés à la gestion manuelle, tout en assurant l'intégrité des données dans un environnement où la gestion des informations volumineuses est cruciale.

Le présent rapport s'articule autour de trois chapitres qui retracent l'enchaînement de la démarche suivie dans la réalisation de ce projet. Je donnerais en un premier temps une description de l'entreprise d'accueil ainsi que le contexte général de mon projet, une élaboration de la problématique et de la méthodologie de gestion de projet adoptée. Le deuxième chapitre met au point la conception du projet et l'architecture technique adoptée pour la réalisation du projet. Et pour mettre en oeuvre tout ce qui précède, je dédie le dernier chapitre à la présentation de la réalisation.

Chapitre 1

Contexte du projet

1.1 Introduction:

Cette partie expose le contexte général du projet, nous allons présenter ses principaux axes : L'organisme d'accueil SOFRECOM où j'ai passé mes 2 mois de stage. Nous allons présenter alors au premier lieu une vue globale sur SOFRECOM, leurs activités, et leur stratégie, ensuite le contexte général du projet, la problématique et les objectifs attendus de notre projet, et on finit par la méthodologie du travail et les étapes de planification du projet.

1.2 Présentation de l'organisme

1.2.1 Présentation de SOFRECOM:

Sofrecom est un cabinet de conseil et d'ingénierie spécialisé dans les télécommunications. Filiale du groupe Orange, Sofrecom conseille et accompagne le développement et la transformation numérique des principaux acteurs du secteur des télécommunications : opérateurs, gouvernements et institutions internationales.

Depuis sa création en 2000, Sofrecom Maroc s'est imposé localement comme acteur incontournable des nouvelles technologies de l'information et de la communication à travers ses activités d'ingénierie logicielle, de conseil et d'ingénierie de réseaux mobiles ainsi que ses activités de veille technologique et concurrentielle.

Le haut niveau d'expertise de ses collaborateurs marocains francophones et la proximité avec l'Europe permettent au groupe d'offrir une flexibilité et une réactivité accrue aux demandes de ses clients.

Filiale du Groupe Orange et spécialiste du secteur des télécommunications, Sofrecom est un acteur unique dans le monde du conseil et de l'ingénierie. Sofrecom analyse et anticipe les tendances du secteur, conseille, accompagne et pilote le développement et les transformations de ses clients. Sofrecom est surtout une entreprise riche de sa diversité avec plus de 2000 consultants et experts issus de plus de 30 nationalités et répartis à travers ses 11 implantations dans le monde.

1.2.2 Principales missions:

Sofrecom accompagne la transformation et le développement des acteurs des télécoms pour construire un monde connecté, créateur de liens.

Sofrecom exerce ses activités dans deux secteurs principaux : Opérateurs télécoms, Gouvernements et institutions publiques.

Elle conseille les opérateurs télécoms sur les 4 continents pour les aider à mieux comprendre leurs clients, à adapter leurs organisations et moderniser leurs réseaux et leurs SI pour atteindre une croissance et une performance durables.

Elle accompagne les gouvernements, les régulateurs et les responsables d'agences de développement numérique pour construire des stratégies numériques facteur de développement économique et d'inclusion.

1.2.3 Domaines d'expertise :

Digitalisation et innovation:

Dans un contexte économique incertain où d'importantes évolutions impactent le monde des télécommunications, analyser les tendances, comprendre les innovations et anticiper les attentes de ses clients sont essentielles pour orienter ses choix stratégiques et développer ses revenus. SOFRECOM aide ses clients à construire leur positionnement, définir leurs stratégies, et à les décliner en plans d'actions opérationnels.

Réseaux et SI:

Les évolutions du réseau et du système d'information sont guidées par les objectifs stratégiques de l'entreprise. Les infrastructures ITN sont des leviers porteurs de valeur et favorisent l'agilité et la performance. SOFRECOM aide ses clients à bâtir une stratégie de transformation réseau et/ou SI au regard de leurs ambitions business et à adapter leurs méthodes et processus pour rester compétitifs. Elle couvre ainsi toutes les étapes d'un projet de modernisation technologique, de la conception d'une stratégie jusqu'à sa mise oeuvre.

Développement durable:

Considéré comme un levier stratégique pour ouvrir de nouvelles opportunités économiques et sociales, l'innovation numérique est un axe majeur des démarches RSE pour réduire l'empreinte environnementale des activités économiques, pour favoriser l'inclusion des populations en anticipant les impacts, en instaurant des synergies entre acteurs. En même temps, le développement durable permet d'améliorer la résilience des entreprises, de créer de la valeur économique. Une approche de développement durable solide requiert donc une stratégie claire, une bonne gouvernance et un reporting d'entreprise de qualité autour des 3 axes : Environnement, Responsabilité sociale et Gouvernance (ESG). SOFRECOM aide ses clients à concilier transformation numérique et développement durable en leur apportant conseil et solutions.

Stratégie Cloud:

Avec l'explosion des données et l'augmentation des cyberattaques, l'hébergement, l'accès et la sécurisation des données deviennent des préoccupations majeures pour les opérateurs télécom, les pouvoirs publics et les entreprises. Les enjeux sont multiples : protection des données sensibles, haute disponibilité, résilience aux attaques, qualité de service, optimisation des coûts, réduction des impacts environnementaux et excellence opérationnelle. SOFRECOM définit à ses clients une stratégie d'hébergement, les aide dans sa déploiement ainsi de concevoir et déployer une infrastructure DATACENTER.

DATA et IA:

Chaque jour, le secteur du télécom génère des quantités considérables de données. Elles sont de tous types, produites sur les réseaux, sur les plateformes de services, dans les canaux de ventes et par les services clients. Ces données sont un capital à exploiter pour générer de la valeur pour les entreprises. Exploiter ces données,

pour en faire un levier de développement de la performance et de l'amélioration de l'expérience clients, nécessite une identification fine des cas d'usage métier, priorisation et planification. Elle implique une préparation à tous les niveaux de l'entreprise pour en tirer les bénéfices en termes de croissance de revenus et d'amélioration de l'expérience client. SOFRECOM aide ses clients à révéler le potentiel de leurs données grâce à l'Intelligence Artificielle.

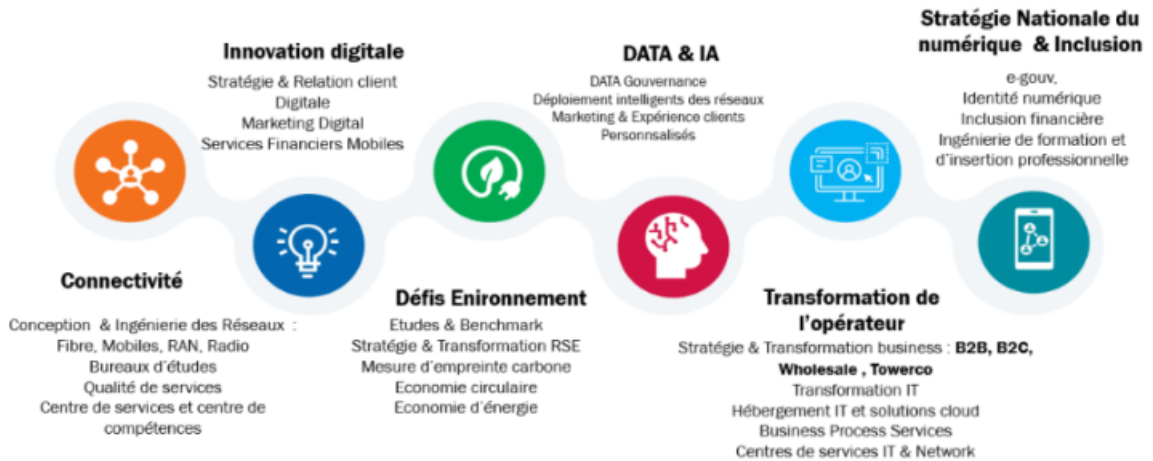


Figure 1.1: domaines d'intervention de Sofrecom Maroc

1.2.4 SOFRECOM en chiffres :

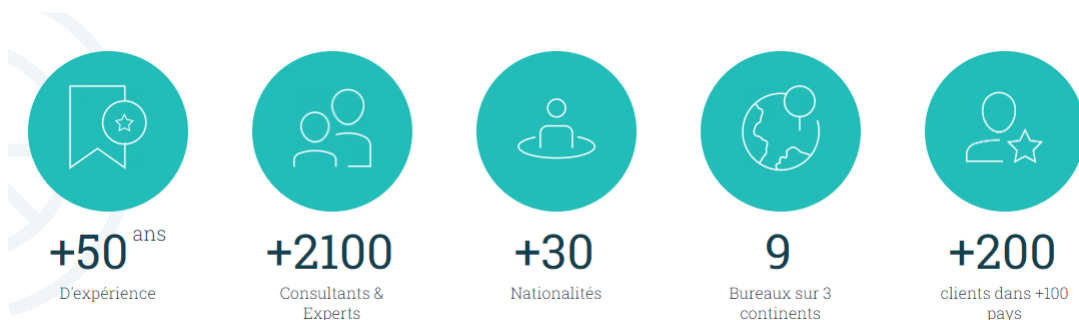


Figure 1.2: Les chiffres clés de SOFRECOM

1.3 Présentation du projet

1.3.1 Problématique du projet:

Dans le domaine de l'infrastructure Big Data sur Google Cloud Platform (GCP), BigQuery s'affirme comme le choix privilégié. Il s'agit d'un entrepôt de données spécialement conçu pour stocker et analyser d'énormes volumes de données en utilisant des requêtes SQL. En parallèle, GCP propose un service cloud géré permettant

d'exécuter des clusters Apache Spark et Apache Hadoop de manière simplifiée et plus efficiente.

Cependant, malgré l'efficacité de BigQuery, les défis surgissent lorsque les data analysts cherchent à récupérer des données complexes à partir de BigQuery via des vues. Ces vues, bien que pratiques, peuvent devenir extrêmement complexes en raison d'autres vues imbriquées et du grand nombre de tables qui les composent. Cela signifie que pour les responsables chargés d'analyser ces données, extraire toutes les dépendances des vues devient un processus laborieux. Parfois, les données doivent être organisées sous forme d'arborescences pour une compréhension approfondie.

Ces défis sont exacerbés par plusieurs facteurs :

- **Défi de la Croissance** : L'adoption rapide de BigQuery a engendré une augmentation exponentielle des structures de données interconnectées, complexifiant les dépendances.

- **Complexité des Autorisations** : Les autorisations deviennent de plus en plus complexes, nécessitant une gestion spécifique pour chaque utilisateur ou groupe.

- **Perte de Temps Chronophage** : Les méthodes manuelles pour déceler les dépendances et configurer les autorisations sont chronophages, exigeant des efforts considérables et de multiples vérifications qui sont source d'erreurs coûteuses.

- **Risques et Conséquences** : Des erreurs dans la gestion des autorisations peuvent causer des violations de sécurité, des perturbations opérationnelles et une perte de confiance dans les données.

De plus, pour accéder à ces tables, les utilisateurs doivent obtenir l'autorisation d'un agent de sécurité afin de pouvoir accéder aux tables BigQuery spécifiques. Ce processus implique généralement de contacter l'agent de sécurité via les systèmes de communication internes de l'entreprise, ce qui n'est pas pratique. La complexité augmente lorsqu'un utilisateur demande l'accès à des centaines de tables regroupées dans une seule vue. Dans de tels cas, donner manuellement l'autorisation par l'intermédiaire de l'agent de sécurité devient un processus laborieux et chronophage. Non seulement cela nécessite une coordination étroite entre les parties concernées, mais cela entraîne également des retards potentiels dans le démarrage des projets d'analyse de données, entravant ainsi la productivité.

1.3.2 Objectifs du projet:

Ce projet a pour objectif de mettre en place un système automatisé visant à gérer efficacement les autorisations d'accès aux tables BigQuery tout en extrayant systématiquement l'arborescence des vues imbriquées et les dépendances entre elles. Cette automatisation est conçue pour améliorer l'efficacité opérationnelle en réduisant les coûts associés à la gestion manuelle. Là où une tâche qui prendrait normalement toute une journée à un individu peut être accomplie en quelques instants par le système, cette automatisation permettra ainsi aux ingénieurs de se concentrer sur le cœur du projet.

Les deux principaux objectifs de ce projet sont les suivants :

-Extraction de l'Arborescence des Vues Imbriquées : Le premier objectif consiste à automatiser le processus d'extraction de l'arborescence des vues imbriquées, y compris la détection des dépendances entre les vues. Cette automatisation permettra de simplifier la compréhension des relations complexes entre les entités et d'accélérer le processus d'analyse des données.

-Gestion Efficace des Autorisations d'Accès : Le deuxième objectif vise à automatiser la gestion des autorisations d'accès aux tables BigQuery. Cette automatisation permettra de garantir l'exécution sécurisée et autorisée des requêtes, éliminant ainsi le besoin de processus manuels fastidieux. Elle contribuera à renforcer la sécurité des données tout en réduisant les erreurs humaines potentielles.

En déléguant ces tâches répétitives au système, l'équipe pourra optimiser son temps et son expertise pour se concentrer sur des aspects plus stratégiques du projet, favorisant ainsi l'efficacité opérationnelle et la qualité des analyses de données.

1.4 Méthodologie du travail

1.4.1 Méthodologie du développement:

Pour garantir une gestion efficace du développement de ce projet, j'ai collaboré étroitement avec mon binôme et notre encadrant. Notre méthodologie de développement s'est appuyée sur l'utilisation de **GitLab** pour la gestion du code source. Cette plateforme nous a permis de maintenir un enregistrement complet de toutes les modifications apportées au code, d'évaluer la qualité du code à travers des revues collaboratives, et de faciliter la collaboration en travaillant de manière isolée sur les fonctionnalités.

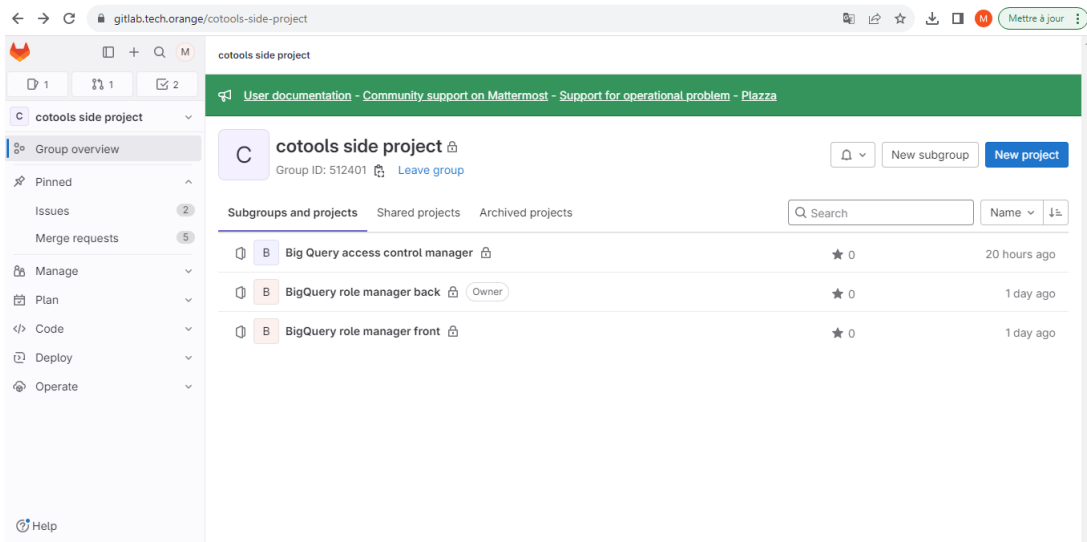


Figure 1.3: Une vue sur la répartition du projet sur Gitlab

1.4.2 Etapes de planification du projet:

La planification du projet revêt une importance capitale, dictant le déroulement à chaque étape du cycle de développement. Il est crucial d’instaurer une organisation appropriée pour orchestrer les tâches de manière efficace. Parmi les outils de gestion de projet, le diagramme de Gantt émerge comme un moyen visuel particulièrement efficace pour suivre l’avancée des activités.

Au cours de la période de stage, le projet est divisé en quatre phases : la phase d’intégration, la phase de montée en compétence, la phase d’étude de projet et enfin, la phase de développement, comme le montre le diagramme de Gantt suivant :

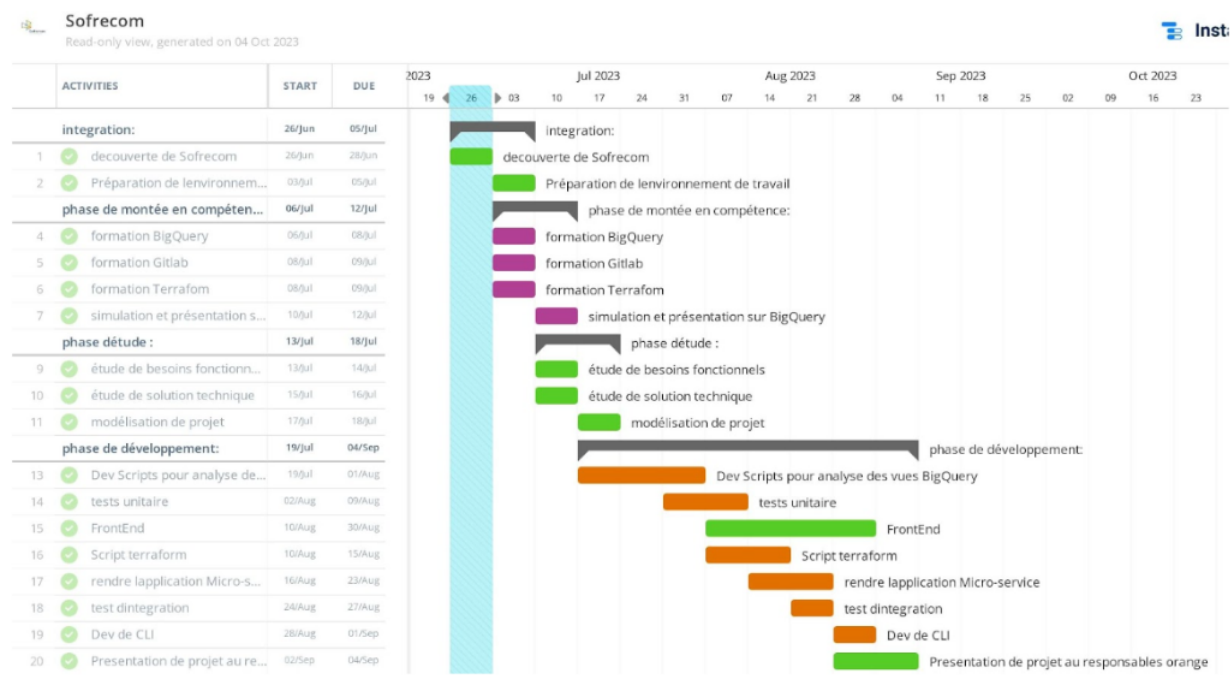


Figure 1.4: Diagramme de Gantt

1.5 Conclusion:

Au terme de ce premier chapitre, nous avons présenté de manière brève le contexte général du projet, la problématique et les objectifs. La première partie a été dédiée à une présentation globale de l’organisme d’accueil SOFRECOM, puis nous avons évoqué le cadre général du projet, la planification et la méthodologie du travail. Ceci facilite l’analyse et la conception qui suivent dans le chapitre suivant.

Chapitre 2

Analyse et conception

2.1 Analyse de l'Existant

Avant la mise en place de notre solution, l'environnement d'accès et de gestion des autorisations dans Google BigQuery au sein de SOFRECOM présentait un certain nombre de limitations importantes. Tout d'abord, il n'existait aucun moyen efficace d'afficher l'arborescence complète des dépendances extraites d'une vue ou d'une requête BigQuery. Les utilisateurs étaient contraints de réexaminer chaque requête source pour identifier toutes les vues et les tables mentionnées, un processus fastidieux et sujet aux erreurs, qui entravait considérablement la compréhension des relations complexes entre les entités de données.

De plus, la procédure d'octroi d'autorisations d'accès aux tables BigQuery était un processus manuel, nécessitant une communication directe entre l'utilisateur demandeur et l'agent de sécurité responsable des autorisations. Les utilisateurs devaient formuler leurs demandes d'accès, expliquer la requête ou la vue spécifique pour laquelle ils avaient besoin d'accès, puis attendre que l'agent de sécurité traite manuellement chaque demande, en ajustant les droits d'accès dans l'interface IAM de Google Cloud pour chaque vue ou table concernée. Cette procédure s'avérait fastidieuse, prenait du temps, et était sujette à des erreurs humaines potentielles.

L'ensemble de ces limitations affectait de manière significative l'efficacité opérationnelle de SOFRECOM, entraînant des retards dans le démarrage des projets d'analyse de données, une utilisation inefficace des ressources humaines et un risque accru d'erreurs dans la gestion des autorisations. Il est devenu évident que des améliorations étaient nécessaires pour optimiser la gestion des autorisations et permettre une visualisation précise des dépendances entre les vues et les tables.

2.2 Spécification des besoins et exigences

La spécification des besoins et des exigences est une étape cruciale dans le processus de développement de notre solution visant à automatiser la gestion des autorisations d'accès aux tables BigQuery et à extraire systématiquement l'arborescence des dépendances des vues. Pour garantir le succès de ce projet, il est essentiel de clarifier en détail ce que la solution doit accomplir. À cet égard, les besoins et les exigences se répartissent en plusieurs catégories clés :

1. Automatisation des Autorisations :

Besoins : La solution doit permettre aux utilisateurs de soumettre des demandes d'accès aux données de manière conviviale. Elle doit automatiser la détection des autorisations requises pour chaque requête BigQuery.

Exigences : La solution doit traiter les demandes d'autorisation en accordant des droits d'accès appropriés aux tables BigQuery spécifiées dans la demande. La gestion des autorisations doit se faire conformément aux meilleures pratiques de sécurité.

2. Visualisation des Dépendances :

Besoins : La solution doit fournir une vue graphique intuitive des relations hiérarchiques entre les autorisations, facilitant ainsi la compréhension des dépendances et la détection rapide de problèmes potentiels.

Exigences : La visualisation des dépendances doit être claire et facile à interpréter, permettant aux utilisateurs de comprendre rapidement les relations entre les vues et les tables. De plus, elle doit être mise à jour en temps réel pour refléter les derniers changements dans la structure des données.

3. Génération de Fichiers JSON :

Besoins : La solution doit permettre aux utilisateurs de générer des fichiers JSON contenant les autorisations et les dépendances pour une utilisation ultérieure.

Exigences : Les fichiers JSON générés doivent être bien structurés et documentés, facilitant leur utilisation dans d'autres applications ou processus.

4. Command Line Interface (CLI) :

Besoins : Une interface en ligne de commande doit permettre aux utilisateurs de générer rapidement les dépendances et d'accorder des droits via Terraform.

Exigences : L'interface en ligne de commande doit être conviviale, offrir des fonctionnalités avancées pour générer des dépendances et gérer les autorisations, et garantir la cohérence avec d'autres méthodes d'accès.

5. Approbation Facilitée :

Besoins : Les examinateurs doivent pouvoir approuver ou rejeter les demandes d'accès, garantissant une gestion fluide des autorisations.

Exigences : Le processus d'approbation doit être transparent, offrir un mécanisme d'audit et assurer que seules les demandes légitimes sont approuvées.

6. Suivi des Demandes :

Besoins : Les utilisateurs doivent être en mesure de suivre l'état de leurs demandes en temps réel, offrant une transparence totale du processus.

Exigences : Le suivi des demandes doit être en temps réel, permettant aux utilisateurs de savoir où en est leur demande et de recevoir des notifications sur les étapes du processus.

7. Traitement Automatisé :

Besoins : Les demandes approuvées doivent être automatiquement traitées pour accorder les droits d'accès nécessaires.

Exigences : Le processus d'accord des droits d'accès doit être automatisé,

minimisant les interventions manuelles et les erreurs potentielles.

2.3 Choix de dataset

Pour tester la faisabilité de notre projet, nous avons reçu un compte Google Cloud Platform (GCP) de l'équipe de Sofrecom avec laquelle nous collaborons. Ce compte GCP nous a permis de créer un environnement de test pour notre solution. Sachant que notre projet visait à automatiser la gestion des autorisations d'accès aux tables BigQuery et à extraire systématiquement l'arborescence des dépendances des vues, nous avons utilisé une approche progressive pour valider notre concept.

Nous avons utilisé un environnement sandbox dans GCP, ce qui nous a permis de travailler dans un environnement contrôlé. Pour alimenter notre MVP et évaluer la fonctionnalité d'extraction de l'arborescence des vues, nous avons opté pour un jeu de données de grande envergure : **le jeu de données COVID-19**. Ce jeu de données complexe comprenait diverses tables et une structure imbriquée.

Au fur et à mesure de notre travail, nous avons créé à la fois des vues simples et des vues imbriquées complexes pour tester la capacité de notre solution à extraire les dépendances entre elles. Cette phase de test nous a permis de vérifier la précision de l'extraction des dépendances et de nous assurer que notre solution fonctionnait conformément à nos objectifs initiaux.

2.4 Stockage dans Cloud Firestore

Dans la phase de conception de notre solution, nous avons soigneusement évalué plusieurs options de stockage de données pour répondre aux besoins spécifiques de notre projet. **Cloud Firestore** de Google Cloud Platform s'est avéré être le choix optimal en raison de plusieurs avantages clés. La flexibilité de sa structure de données nous a permis de modéliser les demandes d'autorisations de manière efficace et conforme à nos exigences.

La structure de stockage que nous avons adoptée repose sur **une collection** principale, **'authorization-requests'**, au cœur de notre architecture. Chaque demande soumise par un utilisateur est représentée comme un document individuel au sein de cette collection. Chacun de **ces documents** contient des informations essentielles telles que l'ID de la demande, le nom du demandeur, l'adresse e-mail du demandeur, la date de la demande, le statut de la demande, le contenu de la demande, des détails supplémentaires, le type de demande, l'adresse e-mail de l'examineur, les commentaires de l'examineur et la date de la décision.

Cette structure nous offre une vue complète de l'historique des demandes, permettant un suivi précis de chaque demande et de son statut actuel. De plus, elle garantit la transparence et la traçabilité du processus de gestion des autorisations, tout en répondant à nos objectifs de sécurité.

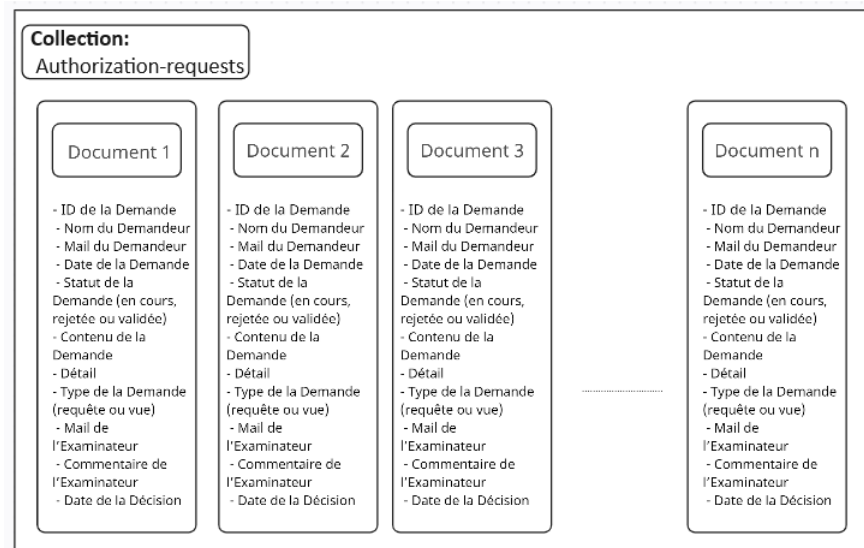


Figure 2.1: Structure du Stockage des demandes dans Firestore

2.5 Diagrammes de Cas d'Utilisation et de Séquence

Nous avons créé des diagrammes de cas d'utilisation et de séquence pour illustrer les interactions au sein de notre système. Ces diagrammes offrent une vue claire des acteurs, des fonctionnalités et des relations clés.

Dans le **diagramme de cas d'utilisation**, nous identifions deux acteurs principaux : **les demandeurs** et **les examinateurs**. Les demandeurs peuvent soumettre une demande d'accès, consulter les détails de leur demande, visualiser les dépendances entre les vues et tables, modifier une demande existante ou la supprimer. Les examinateurs, quant à eux, ont la capacité de lister les demandes en cours, valider une demande ou la rejeter.

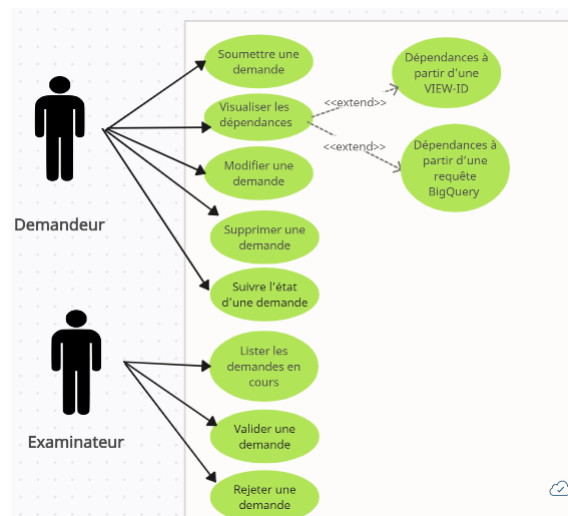


Figure 2.2: Diagramme de cas d'utilisation

Le **diagramme de séquence** pour le cas de visualiser les dépendances illustre comment ces fonctionnalités sont mises en œuvre. Tout commence avec l'acteur du demandeur interagissant avec le système, notamment l'interface graphique, FastAPI, Firestore et Terraform. Lorsqu'un demandeur soumet une demande, cette action est transmise au système via l'interface graphique, qui la transmet à FastAPI pour traitement. Si la demande est validée, elle déclenche Terraform pour l'accord des autorisations.

Les acteurs et le système interagissent de manière similaire pour d'autres fonctionnalités, telles que la visualisation des dépendances. L'interface graphique communique avec FastAPI, qui récupère les données nécessaires dans BigQuery pour générer une réponse que l'utilisateur peut visualiser.

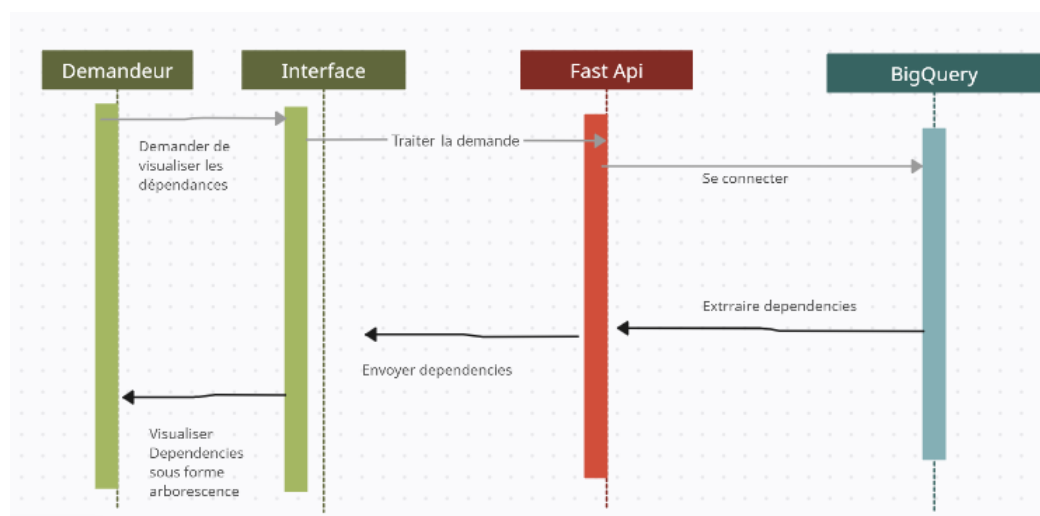


Figure 2.3: Diagramme de séquence: Visualiser les dépendances

En résumé, ces diagrammes illustrent de manière précise les interactions entre les acteurs, le système et les technologies sous-jacentes, offrant une vue complète de notre solution d'accès et de gestion des autorisations pour BigQuery.

2.6 Architecture de la solution:

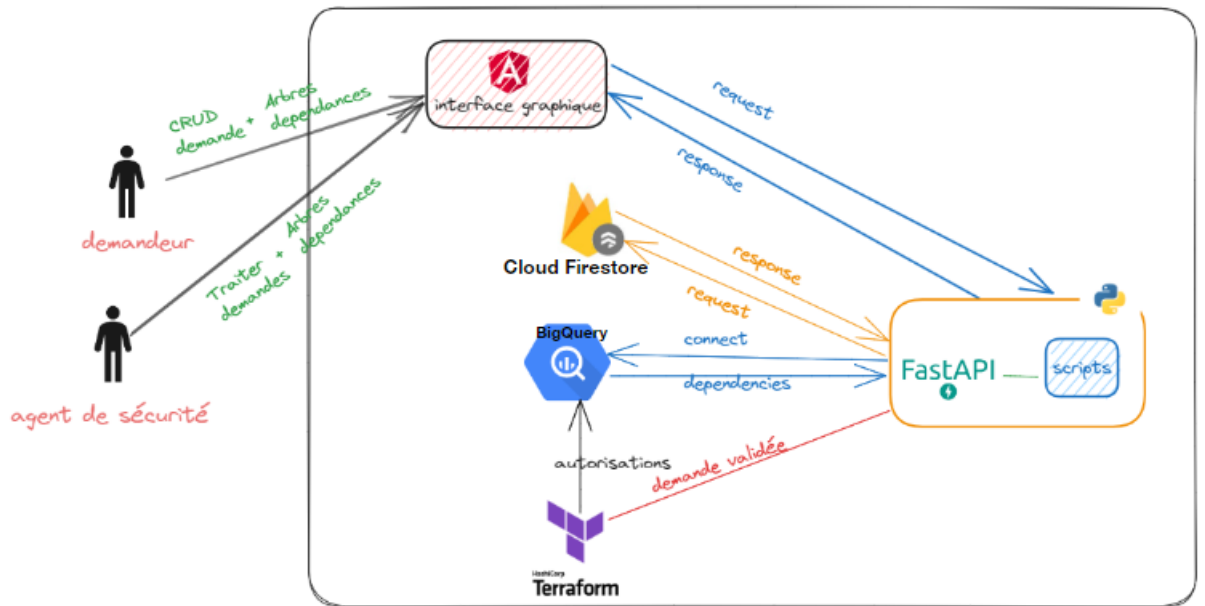


Figure 2.4: Architecture de la solution proposée

Notre architecture repose sur une interaction harmonieuse entre plusieurs éléments clés. Tout démarre avec notre **interface** conviviale, qui offre deux fonctionnalités essentielles : soumettre des demandes d'accès et explorer les dépendances entre les vues et tables de BigQuery.

FastAPI joue un rôle central dans ce processus. Lorsqu'un utilisateur soumet une demande, que ce soit pour accorder des autorisations ou explorer les dépendances, l'interface communique avec FastAPI, qui traite ces requêtes et les achemine vers les microservices appropriés.

Les demandes d'autorisation sont stockées en toute sécurité dans **Firestore**, notre base de données cloud. Cela permet une gestion et un suivi aisés. Si une demande est validée, FastAPI déclenche **Terraform**, notre outil d'automatisation des infrastructures, pour accorder les droits d'accès requis de manière précise et sécurisée.

D'autre part, pour explorer les dépendances, l'interface envoie une requête à **FastAPI** pour obtenir les informations nécessaires. FastAPI se connecte à **BigQuery** via un compte de service configuré, analyse les vues ou les tables spécifiées, puis génère une réponse pour une visualisation claire et intuitive.

Cette architecture garantit une interaction fluide pour la gestion des autorisations et l'exploration des dépendances, offrant ainsi une expérience utilisateur efficace et sécurisée.

Chapitre 3

Réalisation

3.1 Introduction:

Ce chapitre concerne la réalisation de la solution. La première partie sera consacrée à la présentation des technologies et outils utilisés pour la mise en œuvre de notre projet. Nous décrivons, par la suite les étapes de la réalisation et les résultats obtenus.

3.2 Technologies et outils utilisés:

3.2.1 Google Cloud Platform (GCP):



Figure 3.1: Logo Google cloud platform

Google Cloud Platform (GCP) est une suite de services cloud proposée par Google. Elle offre un large éventail de services pour le stockage, le calcul, la gestion des données, l'analyse, l'intelligence artificielle, la gestion des conteneurs, la sécurité, et bien plus encore. GCP permet aux entreprises et aux développeurs d'héberger des applications, de stocker des données et d'utiliser des services cloud de pointe pour répondre à leurs besoins informatiques..

3.2.2 Google BigQuery:



Figure 3.2: Logo BigQuery

Google BigQuery est un service d'entrepôt de données entièrement géré et basé sur le cloud fourni par Google dans le cadre de Google Cloud Platform (GCP). Il est conçu pour stocker, interroger et analyser d'énormes volumes de données à l'aide du langage SQL standard. Il est particulièrement adapté à l'analyse de données en

temps réel et à la manipulation de jeux de données massifs. BigQuery est réputé pour sa grande vitesse et sa facilité d'utilisation.

3.2.3 Cloud Firestore:



Figure 3.3: Logo Firestore

Cloud Firestore est un service de base de données NoSQL entièrement géré proposé par Google Cloud Platform (GCP). Il offre une base de données flexible et évolutive pour le stockage et la gestion des données dans le cloud. Firestore est conçu pour prendre en charge les applications web et mobiles, offrant une synchronisation en temps réel des données entre les clients et la base de données, ce qui permet de créer des applications réactives.

3.2.4 Terraform:



Figure 3.4: Logo Terraform

Terraform est un outil d'infrastructure en tant que code (IaC) qui permet de définir et de gérer l'infrastructure cloud de manière programmable à l'aide de fichiers de configuration. Il prend en charge une variété de fournisseurs de cloud et permet aux équipes de provisionner, de mettre à jour et de surveiller leur infrastructure de manière reproductible. Terraform offre également une planification préalable et des commandes d'application pour garantir un déploiement en toute sécurité.

3.2.5 FastApi:



Figure 3.5: Logo FastApi

FastAPI est un framework Python populaire pour la création d'API RESTful, offrant une syntaxe simple basée sur des types d'annotations. Grâce à sa prise en charge de l'asynchrone et du typage strict, il est largement utilisé pour le développement de **microservices** hautement performants en Python.

3.2.6 CPython:



Figure 3.6: Logo Python

Python est un langage de programmation polyvalent, interprété et de haut niveau, largement utilisé dans le développement de logiciels, la science des données, l'automatisation, le développement web et de nombreuses autres applications.

3.2.7 Angular:



Figure 3.7: Logo Angular

Angular est un framework open-source développé par Google qui est utilisé pour la création d'applications web dynamiques et interactives. Il s'agit d'un framework

JavaScript/TypeScript qui permet aux développeurs de construire des applications côté client robustes et évolutives.

3.3 Etapes de réalisation:

3.3.1 Choix de dataset:

Après la création d'un projet dans Google Cloud Platform (GCP), la prochaine étape a consisté à se tourner vers BigQuery au sein de ce projet.

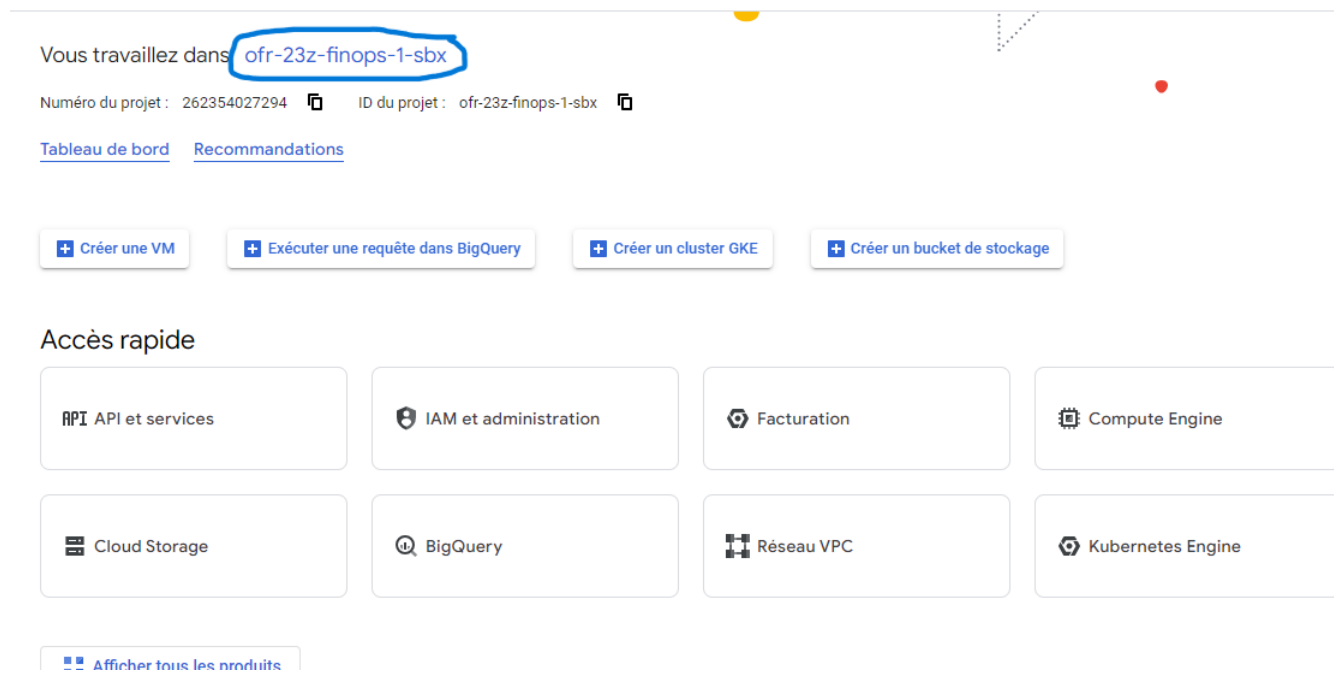


Figure 3.8: Plateforme GCP

Comme précédemment évoqué dans la section de conception, pour tester la faisabilité de notre projet, nous avons bénéficié de l'accès à un environnement de test au sein de Google Cloud Platform (GCP), gracieusement mis à notre disposition par l'équipe de Sofrecom avec laquelle nous collaborons. Cet environnement GCP nous a offert un cadre contrôlé pour valider les concepts et les fonctionnalités que nous voulions développer.

Dans BigQuery, nous avons ajouté un dataset spécifique lié aux données COVID-19. Au sein de ce dataset, nous avons mis en place des tables qui contiennent les données pertinentes pour notre projet.

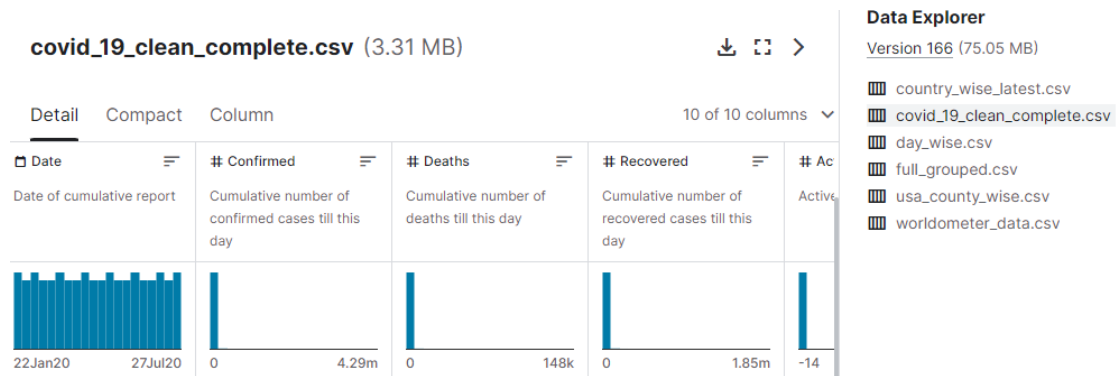


Figure 3.9: Dataset choisie

3.3.2 Création des vues imbriquées:

Par la suite, nous avons entrepris de créer des vues imbriquées qui reflètent la structure complexe des données, Ces vues imbriquées ont été élaborées dans le but de tester notre solution sur divers scénarios, de vues simples à des structures de vue plus complexes, permettant ainsi de tester notre capacité à extraire l'arborescence des dépendances entre ces vues.

Voici un exemple de création d'une vue simple :

```
CREATE VIEW `ofr-23z-finops-1-sbx.CovidDataset.country_summary` AS
SELECT
  c.country,
  c.total_cases,
  c.total_deaths,
  c.total_recovered,
  t.total_tests
FROM
  `ofr-23z-finops-1-sbx.CovidDataset.country_cases` c
JOIN
  `ofr-23z-finops-1-sbx.CovidDataset.country_tests` t
ON
  c.country = t.country;
```

Figure 3.10: Création d'une vue imbriquée

Puis des vues plus complexes avec des jointures entre plusieurs projets et views:

```
CREATE VIEW `ofr-23z-finops-1-sbx.CovidDataset.combined_data` AS
SELECT
  c.country,
  c.total_cases,
  c.total_deaths,
  c.total_recovered,
  t.total_tests
FROM
  `ofr-23z-finops-1-sbx.CovidDataset.country_summary` c
JOIN
  `ofr-23z-finops-1-sbx.CovidDataset.country_tests` t
ON
  c.country = t.country
JOIN
  `ofr-23z-finops-2-sbx.CountryDataset.World-data` w
ON
  c.country = w.Country;
```

Figure 3.11: Création d'une vue plus complexe

Voici une vue sur l'ensemble de données BigQuery après la créations des views .

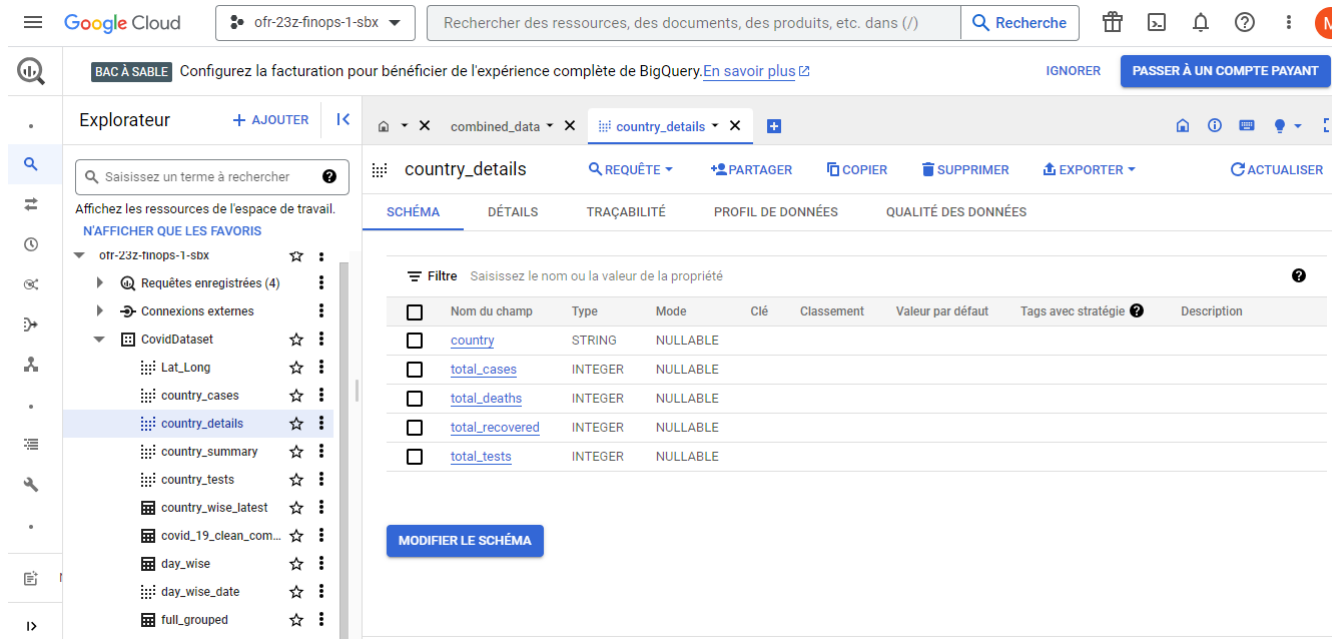


Figure 3.12: Vue sur l'ensemble de données dans BigQuery

3.3.3 Extraction des dépendances:

-Utilisation de SQLParse:

Pour extraire l'arborescence des dépendances des vues, nous avons choisi d'utiliser la bibliothèque Python SQLParse.

SQLParse s'est avéré être un choix judicieux, car il nous a permis d'analyser des requêtes SQL complexes et les views de manière précise et efficace. Voici comment ce module fonctionne :

Validation des Requêtes SQL : Nous avons mis en place des mécanismes de validation pour nous assurer que les requêtes SQL soumises sont valides et détecter rapidement les requêtes incorrectes et de rejeter celles qui ne répondent pas aux normes.

Extraction des Dépendances : La véritable puissance de SQLParse réside dans sa capacité à découper les requêtes SQL en éléments tels que les noms de table, les colonnes, etc. Nous avons exploité cette capacité pour extraire les noms des tables référencées dans chaque requête.

Validation des Tables : Pour garantir la validité des tables référencées on a fait une vérification si la table référencée existe, permettant ainsi d'éviter des erreurs potentielles.

Visualisation des Dépendances : Le résultat de cette extraction est une structure arborescente qui décrit les dépendances des vues mentionnées dans la

requete. Cette structure est stockée au format JSON, offrant une vue claire et structurée de la hiérarchie des dépendances.

Voici un exemple de résultat d'extraction de dépendances d'une vue:

```
{
  "name": "ofr-23z-finops-1-sbx.CovidDataset.country_details",
  "type": "VIEW",
  "dependencies": [
    {
      "name": "ofr-23z-finops-1-sbx.CovidDataset.country_cases",
      "type": "VIEW",
      "dependencies": [
        {
          "name": "ofr-23z-finops-1-sbx.CovidDataset.country_wise_latest",
          "type": "TABLE"
        }
      ]
    }
  ],
  {
    "name": "ofr-23z-finops-1-sbx.CovidDataset.country_tests",
    "type": "VIEW",
    "dependencies": [
      {
        "name": "ofr-23z-finops-1-sbx.CovidDataset.worldometer_data",
        "type": "TABLE"
      }
    ]
  }
]
```

Figure 3.13: Résultat d'extraction des dépendances

-Tests et Couverture du Code:

La fiabilité de notre module d'extraction des dépendances est primordiale. Nous avons développé une suite complète de tests unitaires avec **Pytest** qui couvrent tous les scénarios possibles. Ces tests incluent des **requêtes SQL simples**, des **requêtes complexes**, des **jointures**, des **alias de table**, des **sous-requêtes**, etc. Cette approche garantit que notre module fonctionne correctement dans toutes les situations.

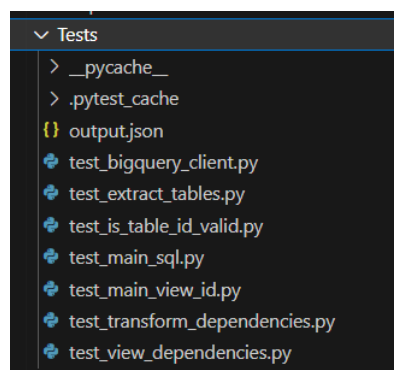


Figure 3.14: Tests unitaires

Pour mesurer l'efficacité de nos tests, nous avons surveillé la couverture du code. Cette métrique nous a permis de garantir que chaque partie du module était correctement testée. Grâce à cette approche rigoureuse, nous avons pu valider notre solution.

Coverage report: 93%				
<i>coverage.py v7.3.0, created at 2023-08-16 15:13 +0100</i>				
Module	statements	missing	excluded	coverage
BigQuery\BigQuery_Client.py	67	1	0	99%
BigQuery\Outils\variables.py	13	0	0	100%
BigQuery\Tests\test_bigquery_client.py	29	1	0	97%
BigQuery\Tests\test_extract_tables.py	38	1	0	97%
BigQuery\Tests\test_is_table_id_valid.py	7	0	0	100%
BigQuery\Tests\test_main_sql.py	65	1	0	98%
BigQuery\Tests\test_main_view_id.py	24	0	0	100%
BigQuery\Tests\test_transform_dependencies.py	20	1	0	95%
BigQuery\Tests\test_view_dependencies.py	30	1	0	97%
BigQuery\Tree_dependencies_SQL.py	29	5	0	83%
BigQuery\Tree_dependencies_ViewId.py	29	10	0	66%
BigQuery__init__.py	14	2	0	86%
BigQuery\tranform_original_file_to_new.py	31	6	0	81%
Total	396	29	0	93%
<i>coverage.py v7.3.0, created at 2023-08-16 15:13 +0100</i>				

Figure 3.15: La couverture du code

3.3.4 Transition vers les Microservices avec FastAPI:

Après la validation de notre module d'extraction des dépendances, nous avons entrepris d'intégrer les autres fonctionnalités de notre solution. Cela incluait **la soumission de demandes** d'avoir des accès aux views, **la gestion des autorisations**, et **le suivi des demandes en temps réel...** nous avons opté pour une transition vers une architecture basée sur des **microservices**, utilisant **FastAPI**.

Cette transition nous a permis de scinder les responsabilités de notre application en modules distincts. Chaque microservice est conçu pour être hautement performant, évolutif et facile à maintenir. FastAPI, en tant que framework Python moderne pour le développement d'API Web rapides, s'est avéré être le choix idéal pour la création de ces microservices.

Nous avons divisé ces microservices en deux parties distinctes : la partie **Demandeur** et la partie **Examineur** de la demande.

Partie Demandeur de la Demande:

La partie Demandeur de la demande englobe plusieurs fonctionnalités clés :

- Générer des Dépendances à partir de Vues ou Requêtes
- Créer une Demande
- Consulter les Détails de la Demande
- Visualiser les Dépendances
- Modifier une Demande
- Supprimer une Demande

Partie Examineur de la Demande:

La partie Examineur de la demande assume un rôle différent mais tout aussi essentiel :

- Lister les Demandes en Cours
- Valider une demande
- Rejeter une Demande

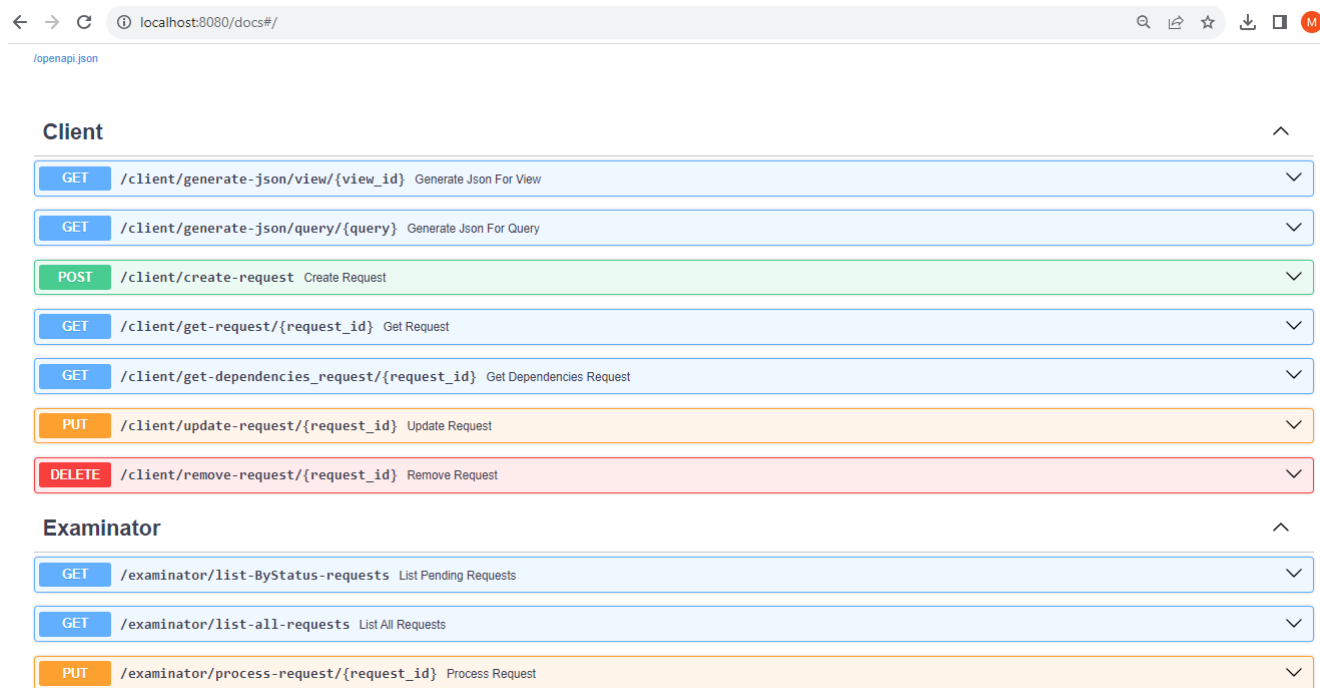


Figure 3.16: Microservices

3.3.5 Stockage des Demandes dans Cloud Firestore:

Comme déjà mentionné dans la conception, Nous avons choisi d'utiliser **Cloud Firestore** pour stocker et gérer les demandes d'autorisations de notre projet

Structure de Stockage:

Le stockage des demandes dans Firestore repose sur une structure de collections et de documents :

- **Collection "authorization-requests"** : Au cœur de notre architecture de stockage, cette collection joue un rôle central. Elle regroupe l'ensemble des demandes soumises par les utilisateurs. Chaque document au sein de cette collection représente une demande spécifique.

Champs des Documents:

Chaque document, c'est-à-dire chaque demande, contient les informations essentielles suivantes :

- **ID de la Demande** : Chaque document est identifié de manière unique par un ID, permettant un accès rapide et précis à une demande particulière.
- **Nom du Demandeur** : Le nom de l'utilisateur qui a soumis la demande, ce qui facilite l'attribution et le suivi des demandes.
- **Mail du Demandeur** : L'adresse e-mail du demandeur est un point de contact clé.
- **Date de la Demande** : Enregistre la date à laquelle la demande a été soumise, offrant une chronologie précise des événements.
- **Statut de la Demande** : Le statut indique si la demande est "en cours", "rejetée" ou "validée". Cette information est cruciale pour le suivi.
- **Contenu de la Demande** : Les détails de la demande, y compris les tables BigQuery ou les vues auxquelles l'accès est requis.
- **Détail** : Informations supplémentaires concernant la demande, permettant une compréhension approfondie des besoins du demandeur.
- **Type de la Demande** : Indique si la demande est basée sur une requête ou une vue, permettant ainsi une gestion différenciée.
- **Mail de l'Examineur** : L'adresse e-mail de l'examineur chargé de l'évaluation de la demande.
- **Commentaire de l'Examineur** : Les examinateurs peuvent ajouter des commentaires pour expliquer leurs décisions.
- **Date de la Décision** : La date à laquelle la demande a été évaluée et que la décision a été prise, importante pour la traçabilité.

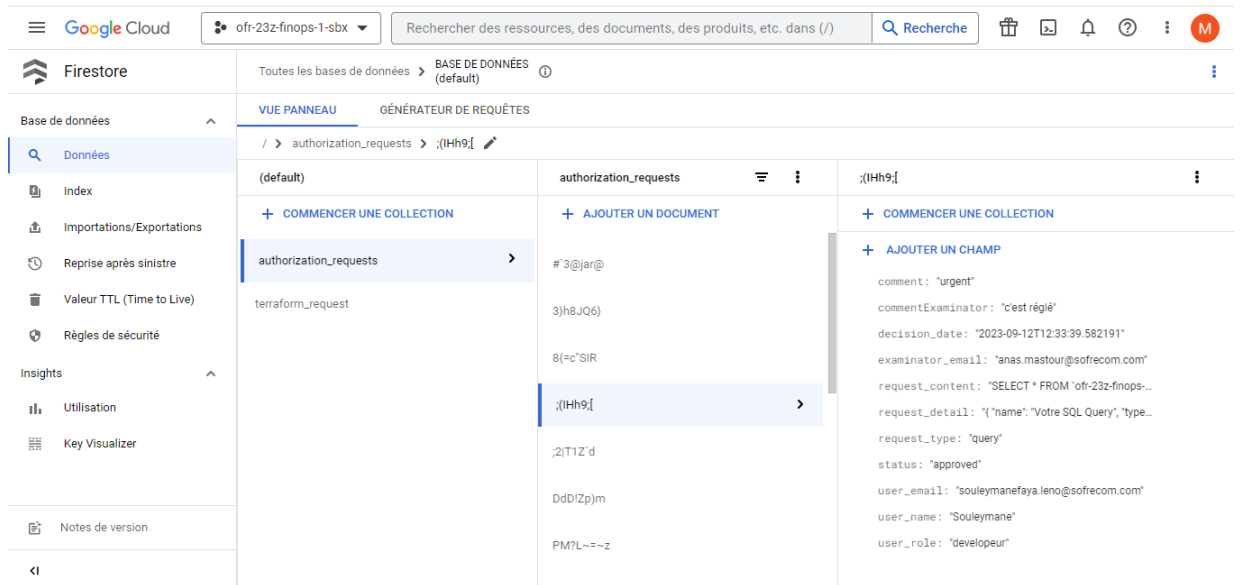


Figure 3.17: Stockage des demandes dans Firestore

3.3.6 Accord des Droits avec Terraform:

Lorsqu'une demande d'accès aux données est approuvée via FastAPI, notre système déclenche **Terraform**, un outil d'automatisation des infrastructures. Terraform joue un rôle clé dans l'accord des droits d'accès requis pour les tables BigQuery spécifiées dans la demande.

Fonctionnement du Processus

Le processus se déroule comme suit :

1. **Récupération de la Demande :** Tout d'abord, le système récupère la demande spécifique depuis Firestore, notre base de données de suivi des demandes. Cela inclut les détails de la demande, tels que les tables ou les vues auxquelles l'accès est demandé.
2. **Mise à Jour de la Demande :** Le statut de la demande est mis à jour en fonction de l'approbation ou du rejet. Les détails de l'examineur, y compris son adresse e-mail et tout commentaire, sont également enregistrés. La date de la décision est ajoutée pour une traçabilité précise.
3. **Récupération des Dépendances :** Les dépendances des tables BigQuery sont extraites des détails de la demande. Cela garantit que Terraform sait quelles tables doivent être accessibles.
4. **Accord des Droits :** Si la demande est approuvée, Terraform procède à l'accord des droits d'accès. Pour chaque table, Terraform se connecte à BigQuery et configure les autorisations. Les utilisateurs autorisés sont ajoutés aux membres autorisés de la table.
5. **Révocation des Droits :** En cas de rejet de la demande, Terraform révoque les droits d'accès. Les utilisateurs ne sont plus autorisés à accéder à ces tables.

```

resource "google_bigquery_dataset_iam_binding" "reader" {
  dataset_id = google_bigquery_dataset.dataset.dataset_id
  role       = "roles/bigquery.dataViewer"

  members = [
    "user:meriem.sabour@sofrecom.com",
  ]
}

resource "google_bigquery_dataset" "dataset" {
  dataset_id = "ofr-23z-finops-2-sbx.CountryDataset.combined_data"
}

```

Figure 3.18: Fichier de Configuration Terraform

Rôle de base	Groupe attribué	Rôle IAM BigQuery
roles/viewer	access.specialGroup: projectReaders	roles/bigquery.dataViewer
roles/editor	access.specialGroup: projectWriters	roles/bigquery.dataEditor
roles/owner	access.specialGroup: projectOwners	roles/bigquery.dataOwner

Figure 3.19: IAM BigQuery roles

Pour faciliter la gestion des droits d'accès par Terraform, nous avons créé une collection Firestore nommée "terraform-request", qui stocke des documents contenant des informations essentielles pour les droits d'accès aux tables BigQuery.

Contenu des Documents

Chaque document au sein de la collection "terraform-request" contient les éléments suivants :

- **project name** : Le nom du projet auquel appartient la table BigQuery concernée.
- **id-dataset** : L'ID du dataset associé à la table.
- **table-id** : L'ID de la table BigQuery spécifique.
- **members** : Une liste des utilisateurs ayant accès à cette table.

Lorsqu'une demande d'accès est approuvée, Terraform utilise ces documents Firestore pour identifier les tables auxquelles les utilisateurs ont accès. Les nouveaux utilisateurs ayant des droits d'accès à une table sont ajoutés à la liste des membres. De même, si une demande est rejetée, Terraform retire le nom de l'utilisateur de la liste des membres, garantissant ainsi la précision des droits d'accès.

Ce processus automatisé, basé sur Terraform, garantit que l'accès aux données est géré de manière précise et sécurisée, en fonction des décisions prises concernant chaque demande individuelle.

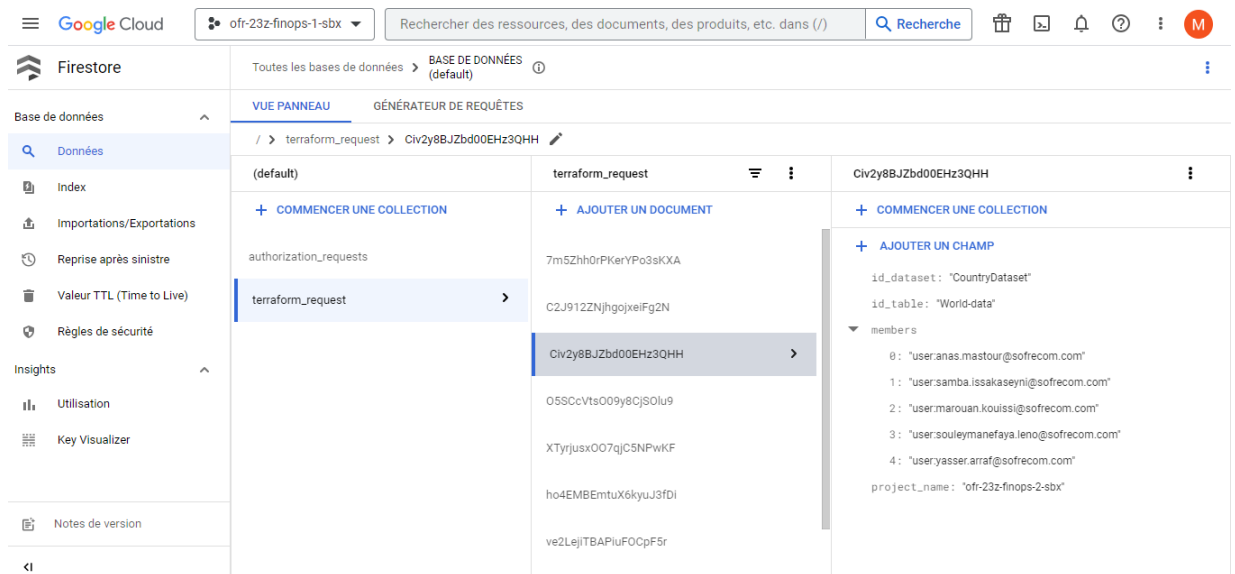


Figure 3.20: Terraform requests

3.4 Interface utilisateur:

Après avoir mis en place notre architecture basée sur des microservices et déployé nos services FastAPI, nous avons développé une interface utilisateur conviviale pour notre solution. Cette interface est construite en utilisant **Angular**, un framework de développement d'applications Web moderne.

L'interface utilisateur offre une expérience fluide aux utilisateurs de notre solution, qu'ils soient des demandeurs ou des examinateurs. Elle permet aux demandeurs de soumettre des demandes d'accès aux données en fournissant des détails tels que les requêtes SQL ou les vues BigQuery dont ils ont besoin. De plus, elle permet aux demandeurs de visualiser l'arborescence des dépendances entre les vues et les tables, offrant une compréhension visuelle instantanée des autorisations requises.

Pour faciliter la gestion des autorisations, l'interface utilisateur permet également de générer un fichier JSON contenant les dépendances. Ce fichier JSON est utilisé ultérieurement dans le processus d'accord des droits via Terraform.

Les examinateurs, de leur côté, peuvent utiliser l'interface pour lister les demandes en cours, les valider ou les rejeter. Cette interface simplifie grandement le processus de gestion des autorisations et offre une visibilité complète sur l'état de chaque demande.

Voici si l'utilisateur veut entrer une view et visualiser les dépendances:

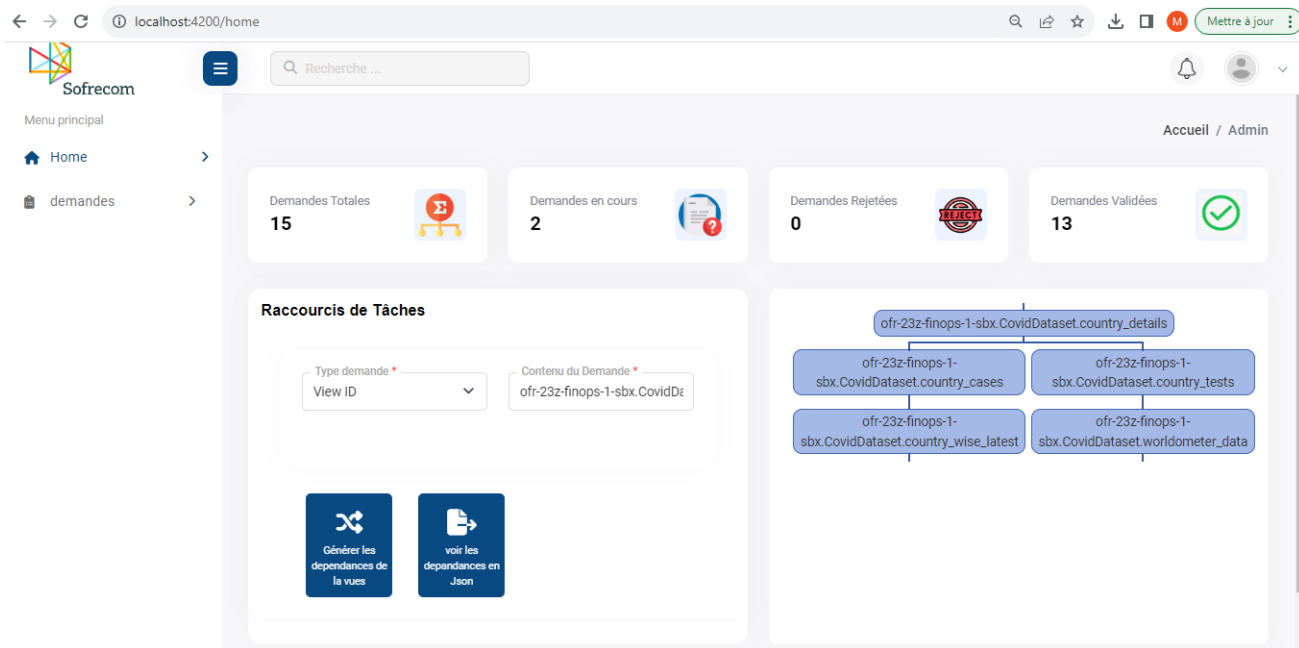


Figure 3.21: L'interface graphique

Le cas d'une requête Big Query:

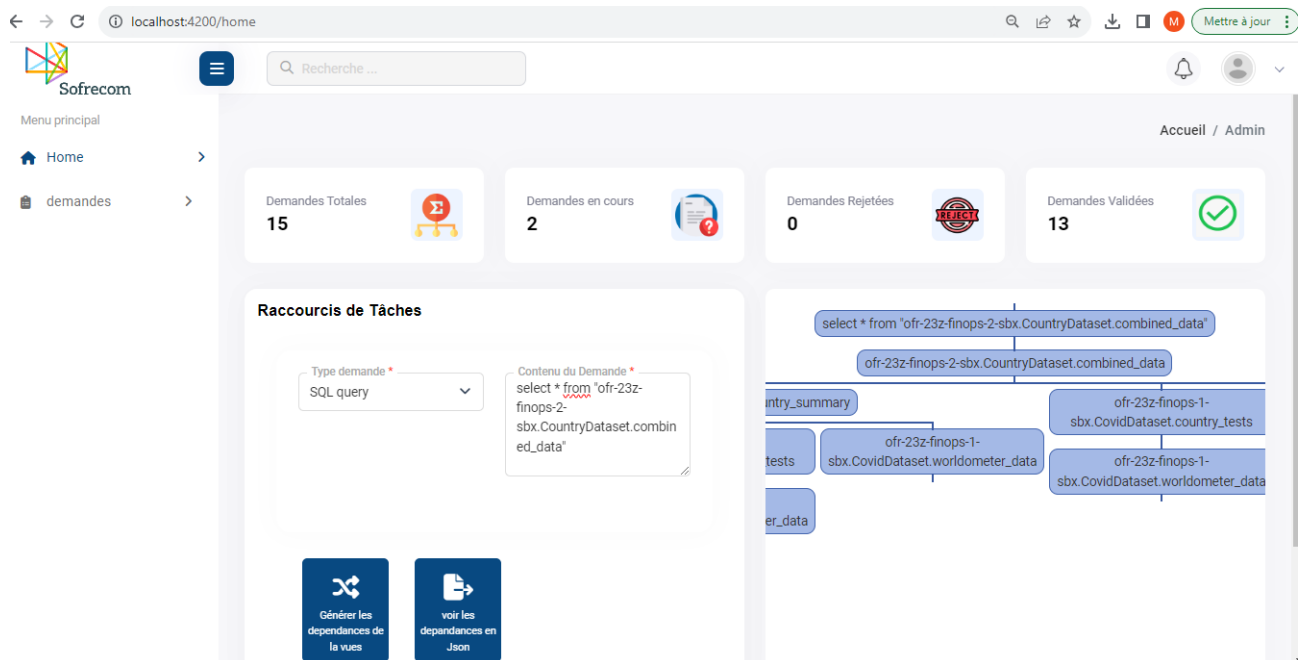


Figure 3.22: L'interface graphique 2

Voici la visualisation générée des dépendances dans le cas d'une view:

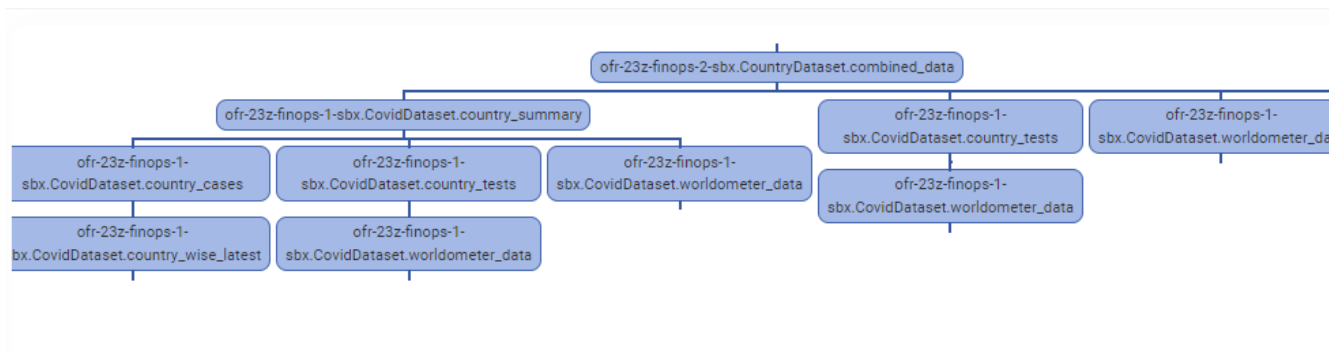


Figure 3.23: Les dépendances d'une vue

Le cas d'une requête Big Query:

Arbre des dependances

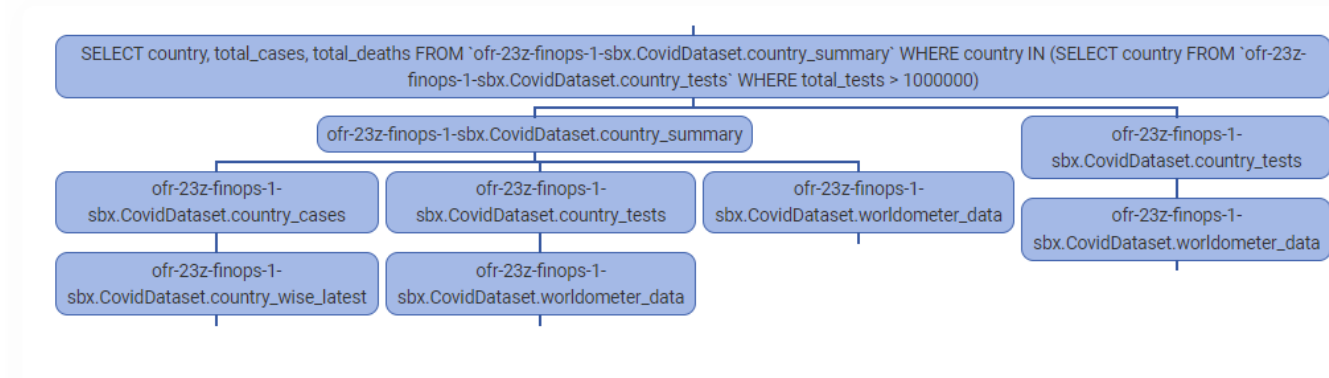


Figure 3.24: Les dépendances d'une requête

Ici l'utilisateur peut soumettre une demande:

The screenshot shows a web browser at the URL `localhost:4200/demandes/add`. The application has a sidebar with the Sofrecom logo and a menu with 'Menu principal', 'Home', and 'demandes'. The main content area is titled 'Demandes / Ajouter un Demande'. It contains a form titled 'Informations sur la demande' with the following fields: 'Nom demandeur *', 'email *', 'poste *', 'Type demande *', 'Contenu du Demande *', and 'Commentaire'. A 'Confirmer' button is at the bottom of the form. The footer of the page reads 'COPYRIGHT © 2023 SOFRECOM.'.

Figure 3.25: Créer une nouvelle demande

Exemple de création d'une demande:

The screenshot shows the same web application but at the URL `localhost:4200/demande/edit`. The form 'Informations sur la demande' is pre-filled with the following data: 'Nom demandeur *' is 'Meriem Sabour', 'email *' is 'meriem.sabour@sofrecom.com', 'poste *' is 'Stagiaire', 'Type demande *' is 'SQL query' (selected from a dropdown), 'Contenu du Demande *' is 'SELECT * FROM `ofr-23z-finops-1-sbx.Covid`', and 'Commentaire' is 'Je veux avoir accès à cette view, C'est urgen'. The 'Confirmer' button is still present. The footer remains 'COPYRIGHT © 2023 SOFRECOM.'.

Figure 3.26: Création d'une demande

Ici l'examineur peut voir toutes les demandes existantes, et filtrer sur les demandes "En cours" pour les valider ou bien les rejeter:

ID demande	Demandeur	email	Date Demande	Status	Action
#'3@jar@	Meriem Sabour	meriem.sabour@sofrecom.com		approved	[plus..]
3)h8JQ6}	samba issaka seyni	samba.issakaseyni@sofrecom.com		approved	[plus..]
8{=c'SIR	samba issaka seyni	samba.issakaseyni@sofrecom.com		approved	[plus..]
;lHh9;[	Souleymane	souleymanefaya.leno@sofrecom.com		approved	[plus..]
DdD!Zp)m	anas mastour	anas.mastour@sofrecom.com		Pending	[plus..]
PM?L~≈~z	anas mastour	anas.mastour@sofrecom.com		approved	[plus..]
SMZpp7=y	Soufiane Sabour	soufiane.sabour@sofrecom.com		approved	[plus..]

Figure 3.27: Une vue sur toutes les demandes

Valider ou bien rejeter une demande :

Traitement de la demande

ID Demande* : 8{=c'SIR

email Examineur* : anas.mastour@sofrecom.com

Commentaire* : c'est ok

Changer Status :

- Pending
- Valider**
- Rejeter

Figure 3.28: Valider une demande

Le cas d'une demande validée:

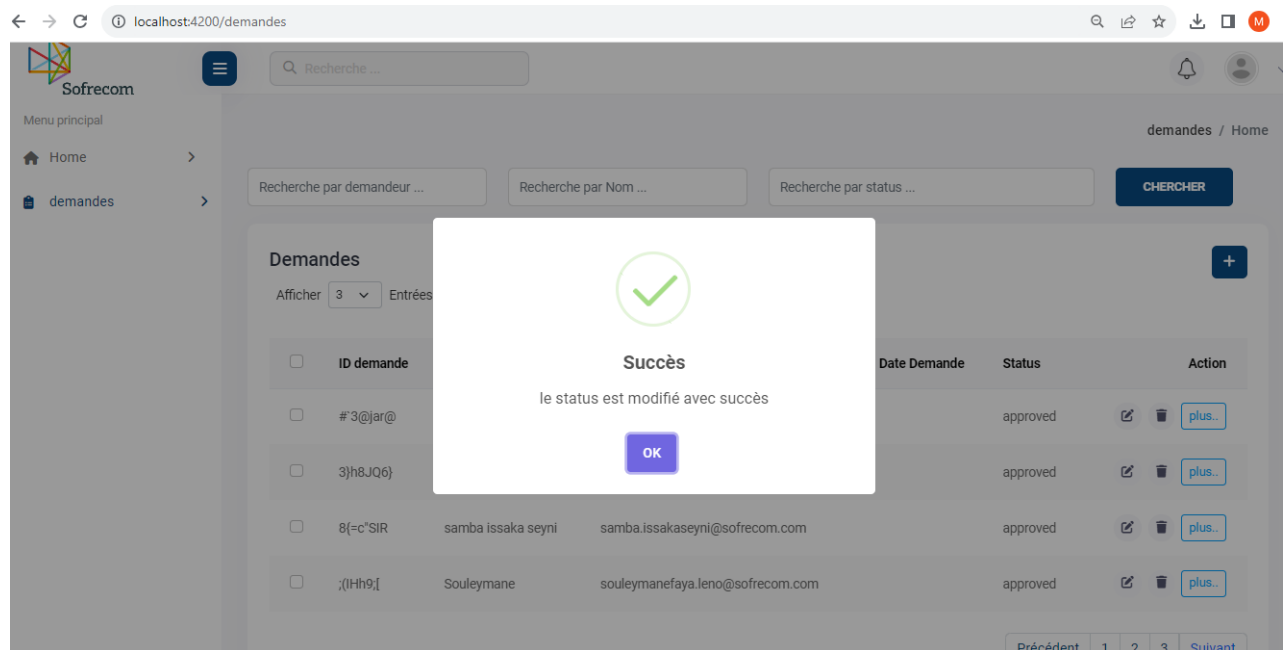


Figure 3.29: Demande validée

La suppression d'une demande:

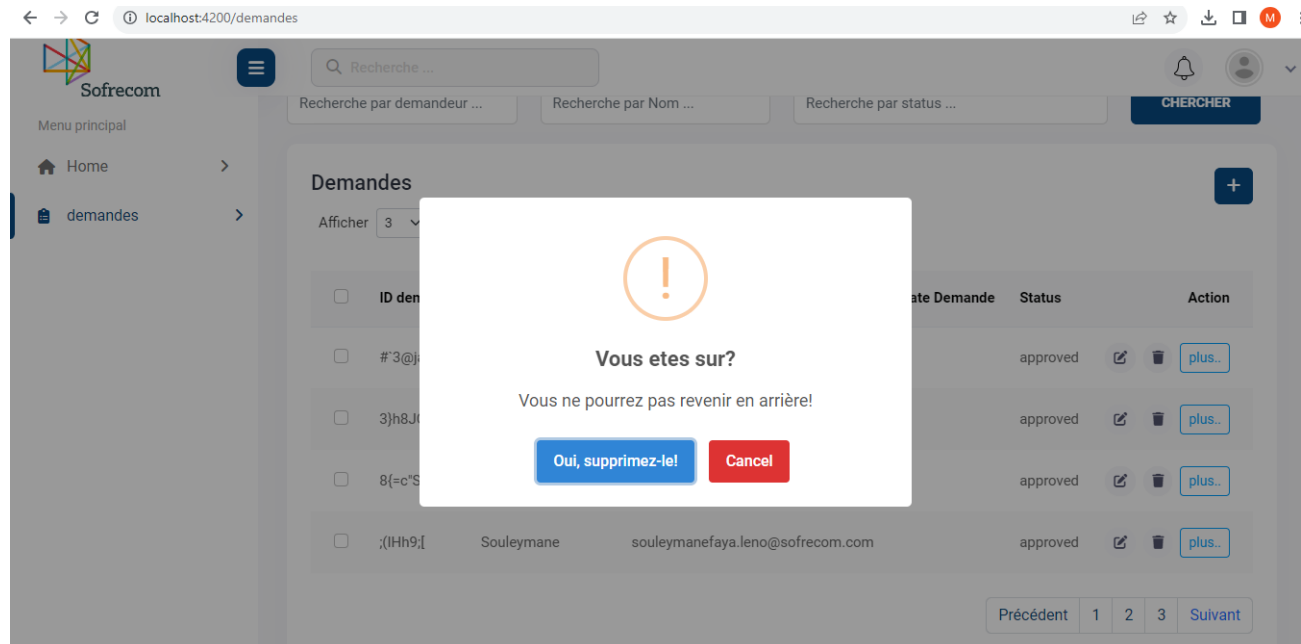


Figure 3.30: Demande supprimée

On revient dans BigQuery et on vérifie bien que les utilisateurs autorisés sont ajoutés aux membres autorisés des tables concernées par la demande

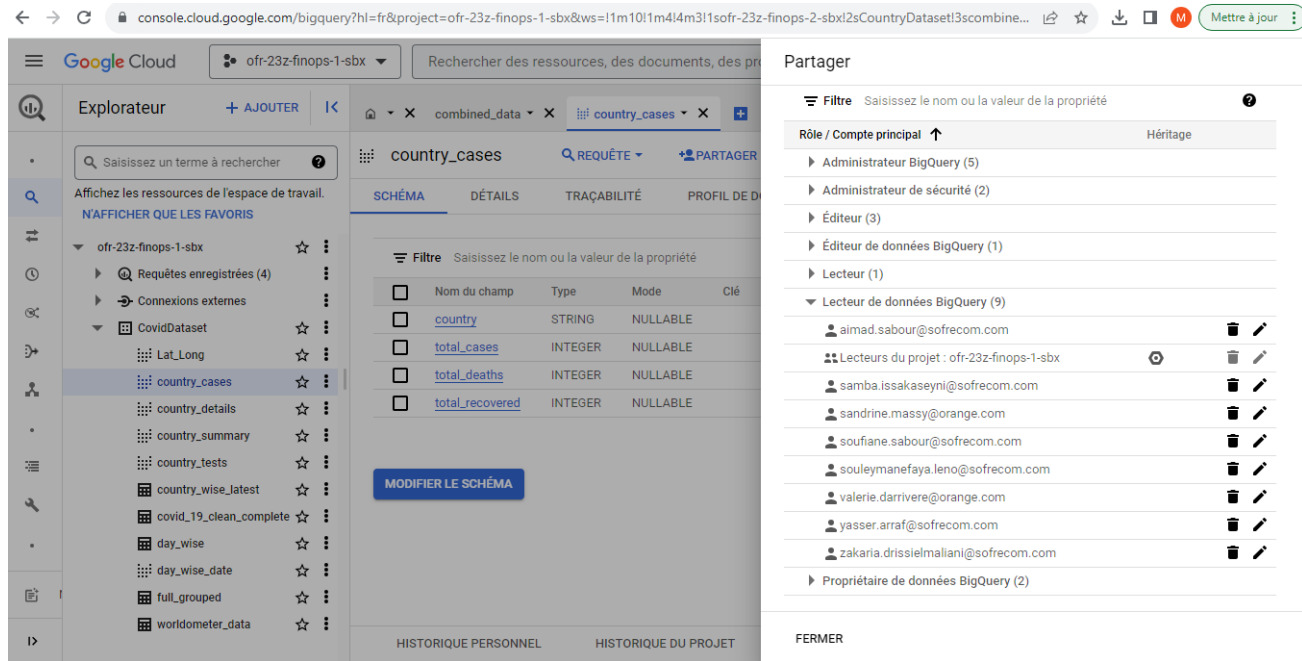


Figure 3.31: Vérification des droits accordés.

3.5 Interface en ligne de commande:

Pour répondre aux besoins des utilisateurs qui préfèrent travailler avec des commandes plutôt qu'avec une interface utilisateur graphique, nous avons développé une interface en ligne de commande (**CLI**) basée sur **Click**, une bibliothèque Python pour la création d'interfaces en ligne de commande.

Cette CLI offre une alternative pratique pour interagir avec notre solution de gestion des autorisations d'accès aux données BigQuery. Elle est conçue pour être simple et intuitive, permettant aux utilisateurs de générer des fichiers JSON contenant des dépendances de vue et d'exécuter des opérations de manière efficace.

Les fonctionnalités clés de notre CLI comprennent :

- **Génération de JSON** : Les utilisateurs peuvent générer des fichiers JSON contenant les dépendances des vues ou requêtes BigQuery. Cela est utile pour fournir des informations essentielles à Terraform pour accorder les autorisations nécessaires.

- **Options Flexibles** : Les utilisateurs ont la possibilité de fournir des informations telles que le **chemin des informations d'identification**, le **fichier de sortie**, le choix entre la spécification d'une vue BigQuery via une **requête SQL** ou un **identifiant de vue**.

Pour utiliser la CLI, les utilisateurs doivent simplement exécuter le script avec les options appropriées, comme le chemin des informations d'identification, le type de requête, et les informations de sortie souhaitées. La CLI traitera ensuite les demandes en fonction des options spécifiées.

Notre package de ligne de commande (CLI) a été soigneusement nommé **bqacm**, abréviation de "**BigQuery Access Control Management**".

Lorsque l'utilisateur exécute la commande **bqacm --help**, la CLI affiche un ensemble d'options clairement définies. Les options telles que **le chemin des informations d'identification**, **le fichier de sortie JSON**, la génération de fichiers JSON pour Terraform, **l'identifiant de vue BigQuery**, ou **la requête SQL** sont toutes documentées et expliquées pour faciliter leur utilisation.

```

• (pfa) amastour@workspace:~/workspace/python/bqacm$ bqacm --help
Usage: bqacm [OPTIONS]

Command-line interface for generating JSON files based on user options.

Args:
  credentials_path (str): Path to the credentials file.
  view_id (str): The view ID.
  output_file (str): Path to the output file.
  generate_json (bool): Whether to generate JSON.
  sql_query (bool): Option V.
  view_id (bool): Option D.

Options:
  -c, --credentials_path TEXT  Path to the credentials file. This is the path
                                to the service account key file. Make sure to
                                set the path of this file in your system's
                                environment variables under the name
                                'CREDENTIALS_PATH'. It is important to save it
                                with this name, otherwise it will not work.
  --output_file TEXT           Path to the output JSON file. This is the
                                location where you want to store the result of
                                the processing. The file will contain the
                                dependency tree of the given view.
  -t, --generate_json          Generate a file to provide Terraform with
                                necessary permissions. Use this option to
                                generate the file for Terraform usage.
  -v, --view_id TEXT           This option 'v' generates dependencies of the
                                view using the view ID. You must choose either
                                -q or -v.
  -q, --sql_query TEXT         This option '-q' generates dependencies of the
                                view using a SQL query. You must choose either
                                -q or -v.
  --help                       Show this message and exit.
○ (pfa) amastour@workspace:~/workspace/python/bqacm$

```

Figure 3.32: Options Bqacm.

Exemple de commande pour extraire les dépendances d'une view:

```
(pfa) amastour@workspace:~/workspace/python/bqacm$ bqacm -v ofr-23z-finops-1-sbx.CovidDataset.country_details -c /workspace/python/gestion_autori
JSON file successfully generated with view dependencies.
(pfa) amastour@workspace:~/workspace/python/bqacm$ cat output.json
{
  "name": "ofr-23z-finops-1-sbx.CovidDataset.country_details",
  "type": "VIEW",
  "dependencies": [
    {
      "name": "ofr-23z-finops-1-sbx.CovidDataset.country_cases",
      "type": "VIEW",
      "dependencies": [
        {
          "name": "ofr-23z-finops-1-sbx.CovidDataset.country_wise_latest",
          "type": "TABLE"
        }
      ]
    }
  ],
  {
    "name": "ofr-23z-finops-1-sbx.CovidDataset.country_tests",
    "type": "VIEW",
    "dependencies": [
      {
        "name": "ofr-23z-finops-1-sbx.CovidDataset.worldometer_data",
        "type": "TABLE"
      }
    ]
  }
]
}
(pfa) amastour@workspace:~/workspace/python/bqacm$
```

Figure 3.33: Générer dépendances d'une view.

Exemple de commande pour extraire les dépendances d'une requete BigQuery:

```
(pfa) amastour@workspace:~/workspace/python/bqacm$ bqacm -q 'select * from "ofr-23z-finops-2-sbx.CountryDataset.combined_data"' -c /workspace/python/gestion_autorisations_bigquery/Assets/Service
output.json
select * from "ofr-23z-finops-2-sbx.CountryDataset.combined_data"
JSON file successfully generated with view dependencies.
(pfa) amastour@workspace:~/workspace/python/bqacm$ cat output.json
{
  "name": "select * from \"ofr-23z-finops-2-sbx.CountryDataset.combined_data\"",
  "type": "VIEW",
  "dependencies": [
    {
      "name": "ofr-23z-finops-2-sbx.CountryDataset.combined_data",
      "type": "VIEW",
      "dependencies": [
        {
          "name": "ofr-23z-finops-1-sbx.CovidDataset.country_summary",
          "type": "VIEW",
          "dependencies": [
            {
              "name": "ofr-23z-finops-1-sbx.CovidDataset.country_cases",
              "type": "VIEW",
              "dependencies": [
                {
                  "name": "ofr-23z-finops-1-sbx.CovidDataset.country_wise_latest",
                  "type": "TABLE"
                }
              ]
            }
          ]
        }
      ]
    },
    {
      "name": "ofr-23z-finops-1-sbx.CovidDataset.country_tests",
      "type": "VIEW",
      "dependencies": [
        {
          "name": "ofr-23z-finops-1-sbx.CovidDataset.worldometer_data",
          "type": "TABLE"
        }
      ]
    },
    {
      "name": "ofr-23z-finops-1-sbx.CovidDataset.worldometer_data",
      "type": "TABLE"
    }
  ],
  {
    "name": "ofr-23z-finops-1-sbx.CovidDataset.country_tests",
    "type": "VIEW",
    "dependencies": [

```

Figure 3.34: Générer dépendances d'une query.

3.6 Conclusion:

Nous avons exploré dans ce chapitre les technologies et outils clés, notamment Google Cloud Platform, BigQuery, FastAPI, Terraform, Cloud Firestore et Click. En détaillant les étapes de développement, de la création de microservices à la gestion des droits avec Terraform, en passant par la conception d'interfaces utilisateur conviviales, Cette approche polyvalente, alliant simplicité d'utilisation et précision, promet de simplifier considérablement la gestion des autorisations d'accès aux données dans notre environnement BigQuery.

Conclusion générale

Durant mes deux mois de stage chez SOFRECOM, j'ai eu l'opportunité exceptionnelle de travailler sur un projet d'une envergure significative. Ce projet, intitulé "**Explorateur de Dépendances BigQuery : Gestion des Autorisations et Visualisation des Dépendances**" m'a permis de plonger au cœur de l'écosystème **Google Cloud Platform (GCP)** et d'explorer en profondeur les services offerts par Firestore, BigQuery et Terraform.

La problématique à laquelle ce projet visait à répondre était complexe : comment simplifier la gestion des autorisations d'accès aux données et comment visualiser de manière claire les dépendances entre les vues et les requêtes dans Google BigQuery ? Ces défis étaient exacerbés par la nature en constante évolution de l'environnement BigQuery, qui exigeait une solution robuste pour garantir la sécurité et la conformité des données.

En collaboration avec mon binôme, nous avons développé une solution complète qui permet de soumettre des demandes d'accès aux données, de visualiser graphiquement les dépendances, de générer des fichiers JSON contenant les autorisations et les dépendances, et d'automatiser la gestion des dépendances et l'accord des droits via Terraform, et une option de CLI en utilisant des commandes. Cette solution a été conçue pour être hautement performante, évolutive et conviviale, avec l'objectif de simplifier la gestion des autorisations et de renforcer la sécurité des données.

Mon parcours chez Sofrecom m'a incité à approfondir ma compréhension des défis liés à la sécurité et à la gestion des données dans un environnement en constante évolution. J'ai acquis des compétences pratiques dans l'utilisation d'outils et de services de pointe pour répondre aux besoins de l'entreprise en matière de gestion des données et de sécurité.

En ce qui concerne les perspectives, je suis déterminé à continuer d'apprendre et de progresser dans la sécurité de big Data et le Cloud. Je souhaite également explorer des possibilités d'amélioration continue de la solution, notamment en ajoutant des fonctionnalités pour accorder les droits par Terraform via l'interface en ligne de commande (CLI). Si le temps et les ressources le permettent, je suis ouvert à l'extension de cette solution pour répondre à des besoins complexes et en constante évolution.

Cette expérience a été riche en enseignements, et je suis reconnaissant envers toute l'équipe qui m'a soutenu et guidé tout au long de mon stage. Elle a renforcé ma détermination à poursuivre mon développement professionnel dans le domaine de Data Engineering et CLOUD.

Webographie

<https://www.kaggle.com/datasets/imdevskp/corona-virus-report>

<https://cloud.google.com/bigquery/docs/access-control?hl=fr>

<https://www.sofrecom.com/>

<https://www.merixstudio.com/blog/how-use-fastapi-microservices-python/>

https://docs.oracle.com/fr-fr/iaas/Content/partner-portal/partner-portal_build-create_erraform_configuration_files.htm